

Linguagem C e sistemas embarcados



Programação de periféricos

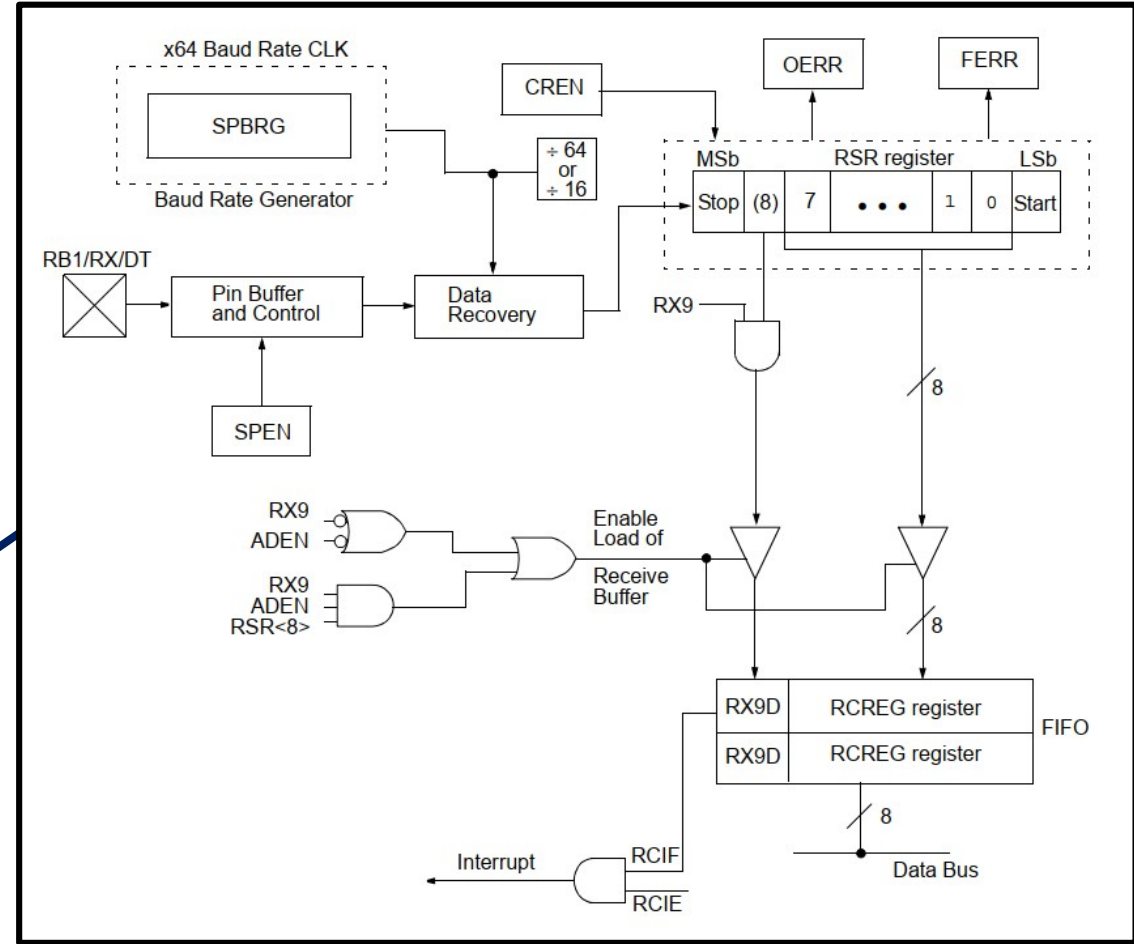
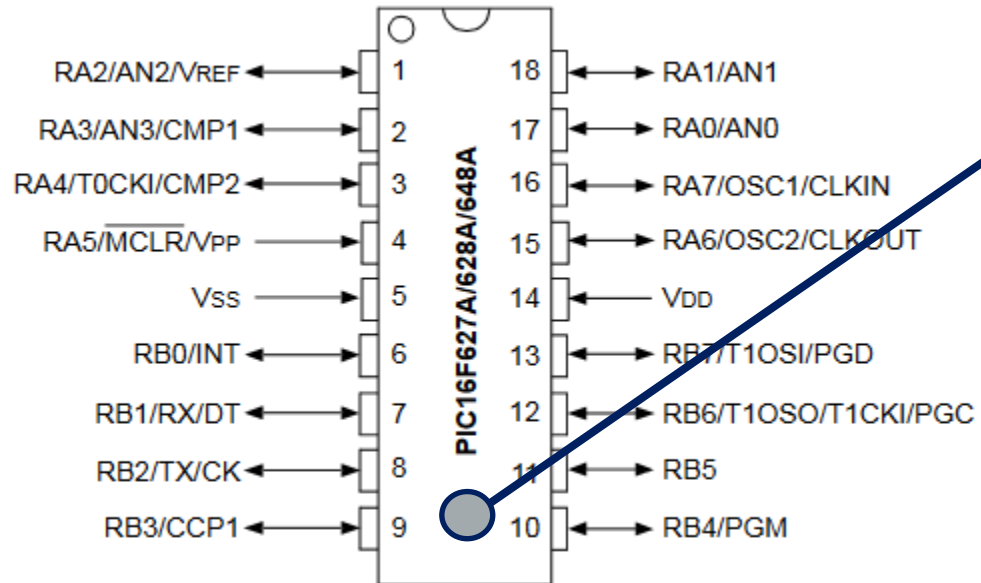


Periféricos

- Circuitos que podem executar atividades dedicadas de modo independente do processador.
- Exemplos:
 - Conversor de sinal analógico em digital.
 - Contador de tempo.
 - Comunicação serial.
 - Portas de comunicação.



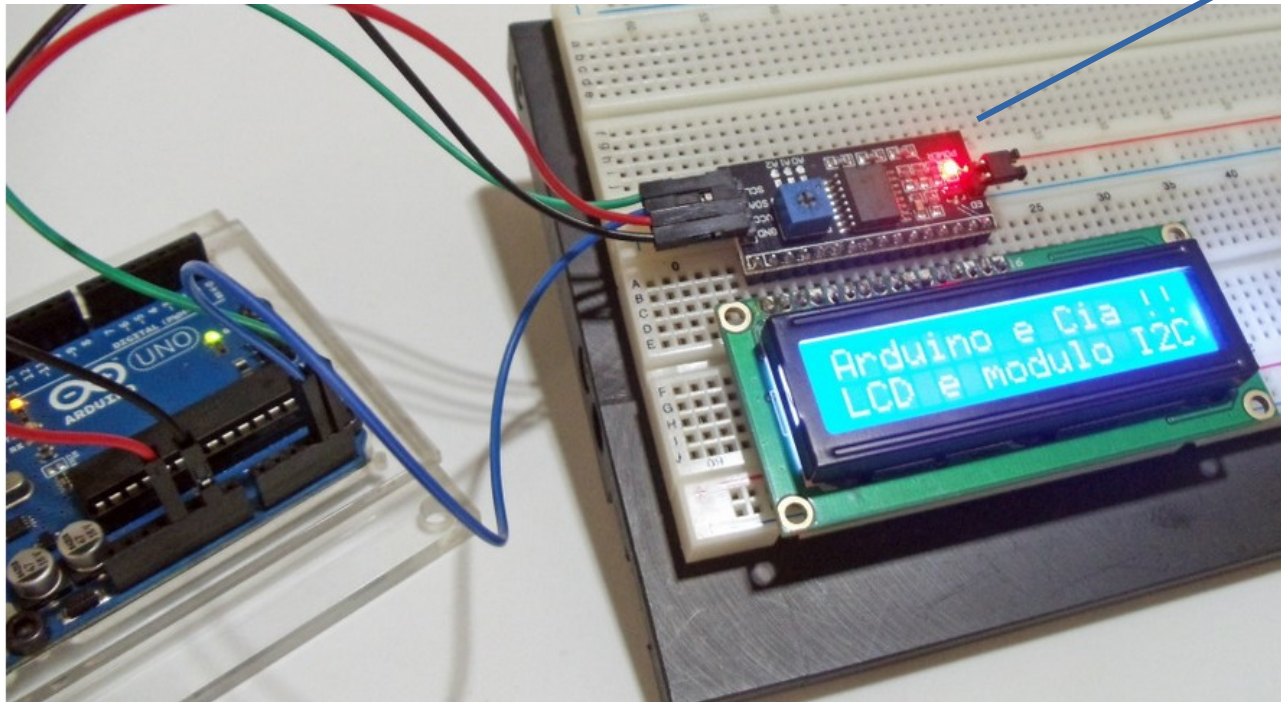
Periféricos Internos



Comunicação Serial

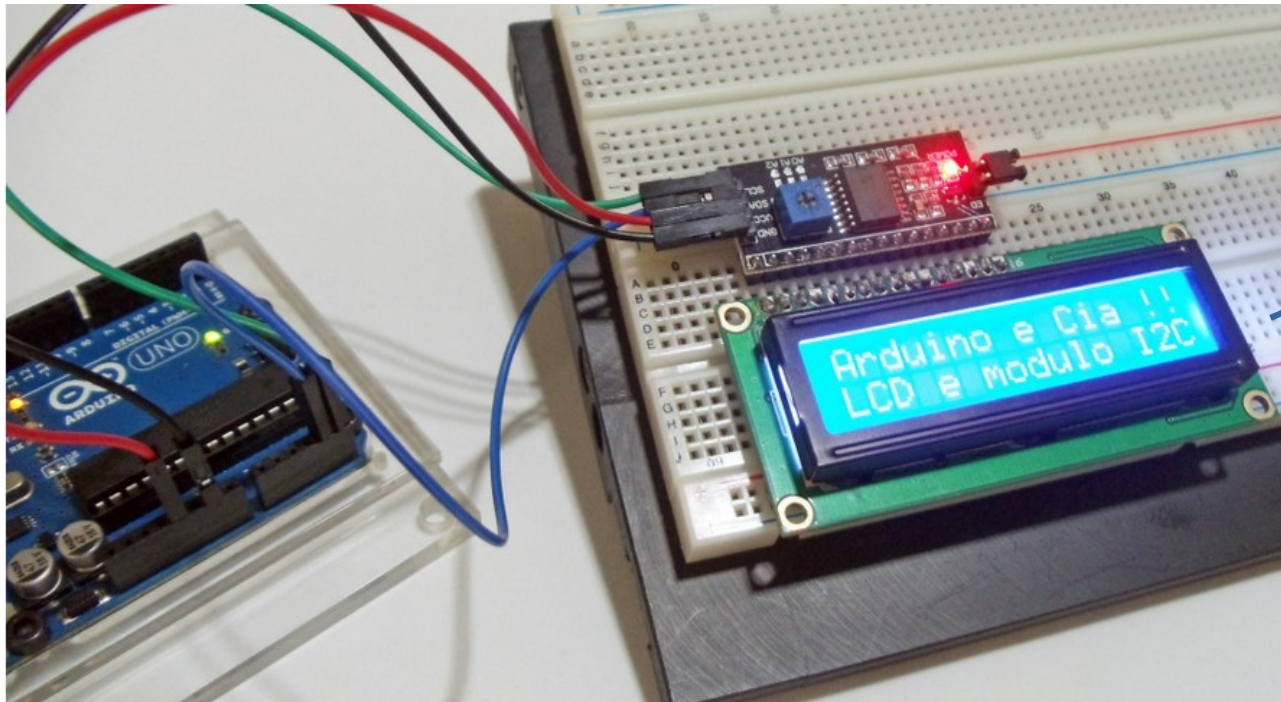
Periféricos Externos

Módulo I2C



Periféricos Externos

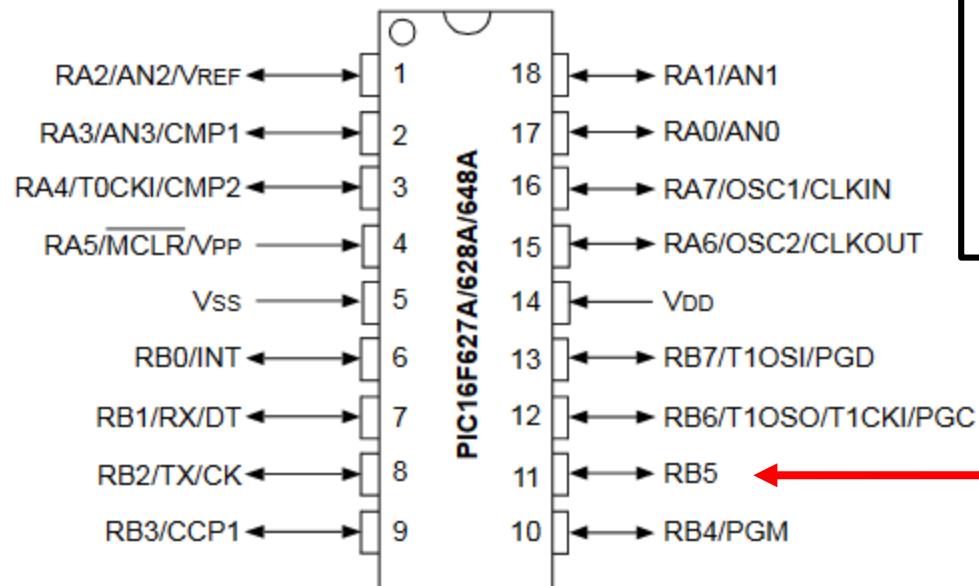
Periféricos externos como display e teclado possibilitam uma comunicação direta com o usuário.



Display

Periféricos de entrada/saída

- As portas são circuitos que repassam a informação da memória para os terminais físicos.

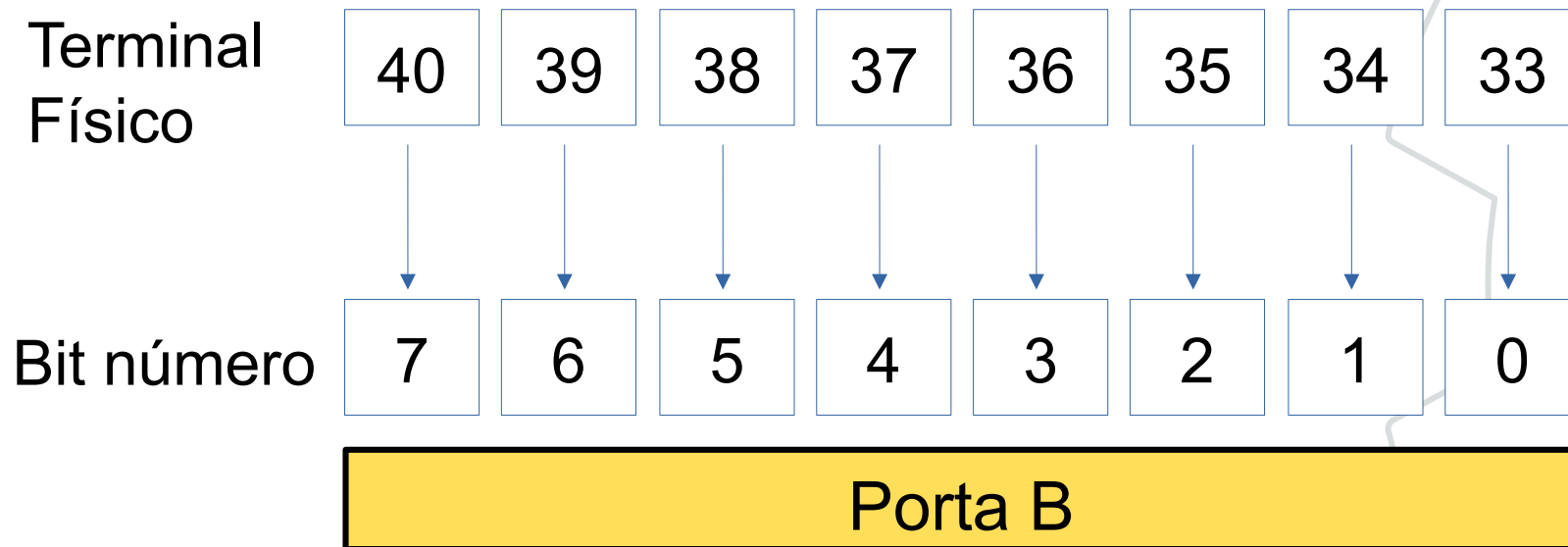


Neste exemplo
Cada bit do registrador (memória)
representa o estado de um terminal.

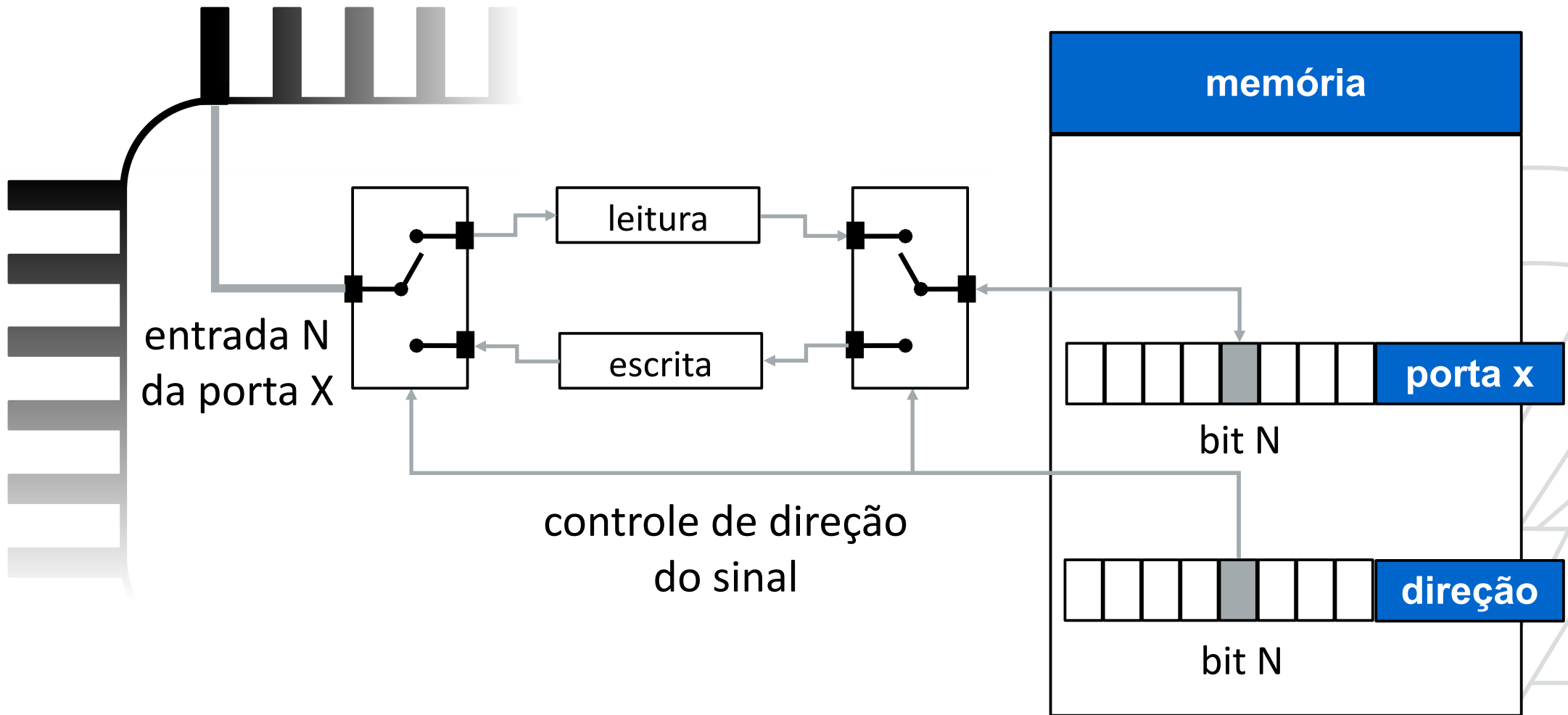
Pino 11, porta B, bit 5

Periféricos de entrada/saída

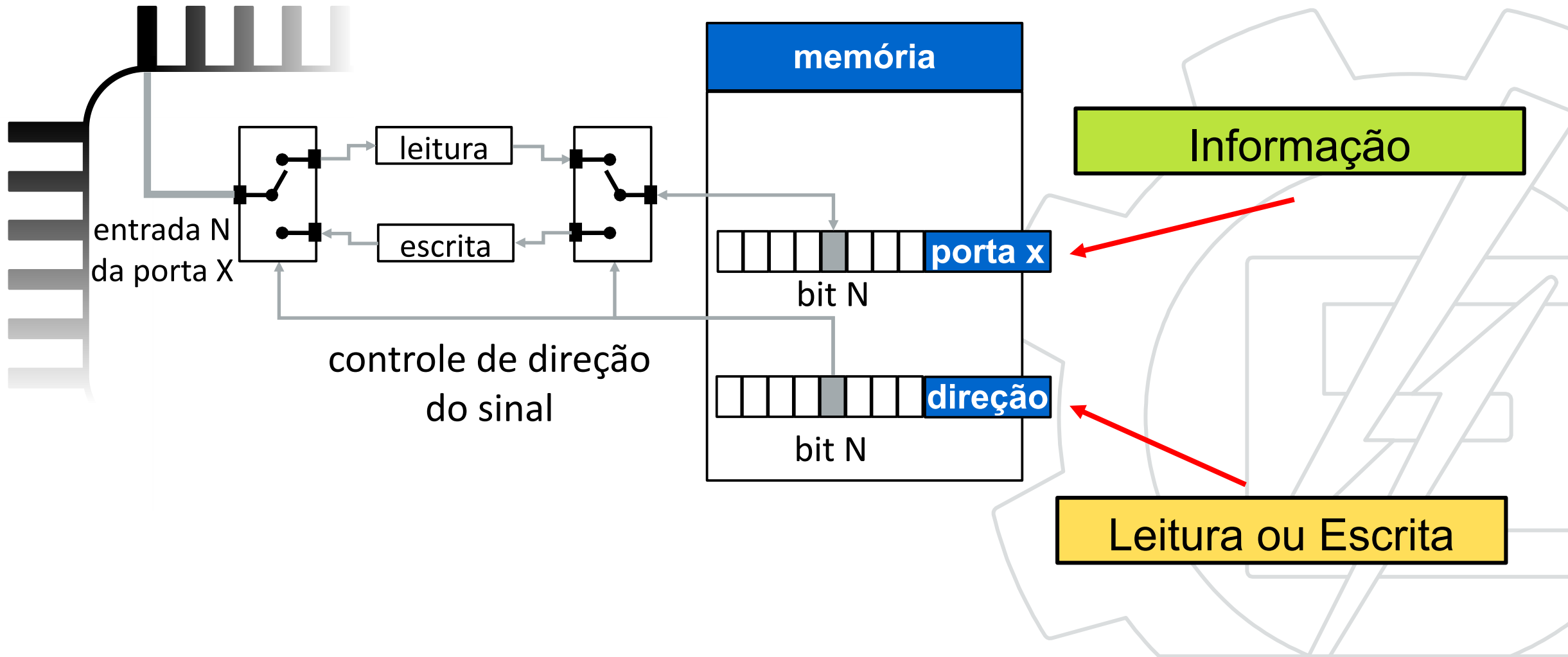
- Exemplo
 - Os oito bits (0 a 7) estão diretamente conectados aos terminais físicos de número 33 a 40.



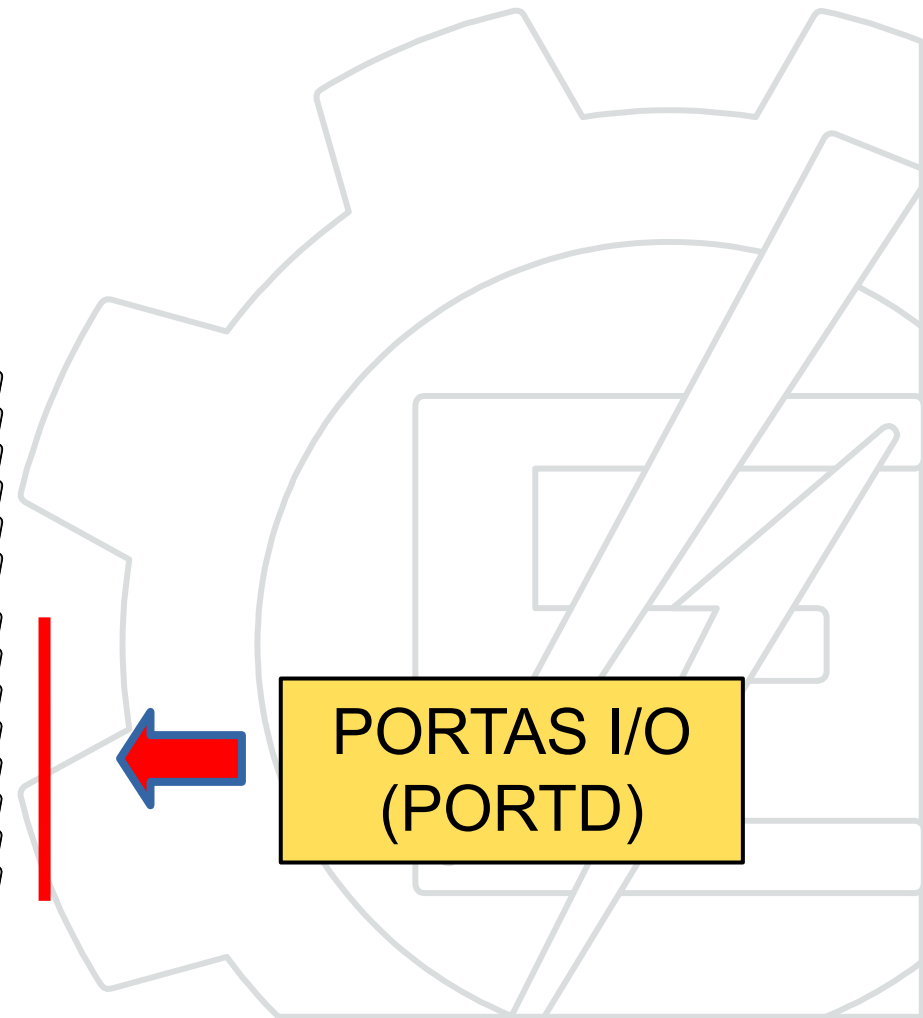
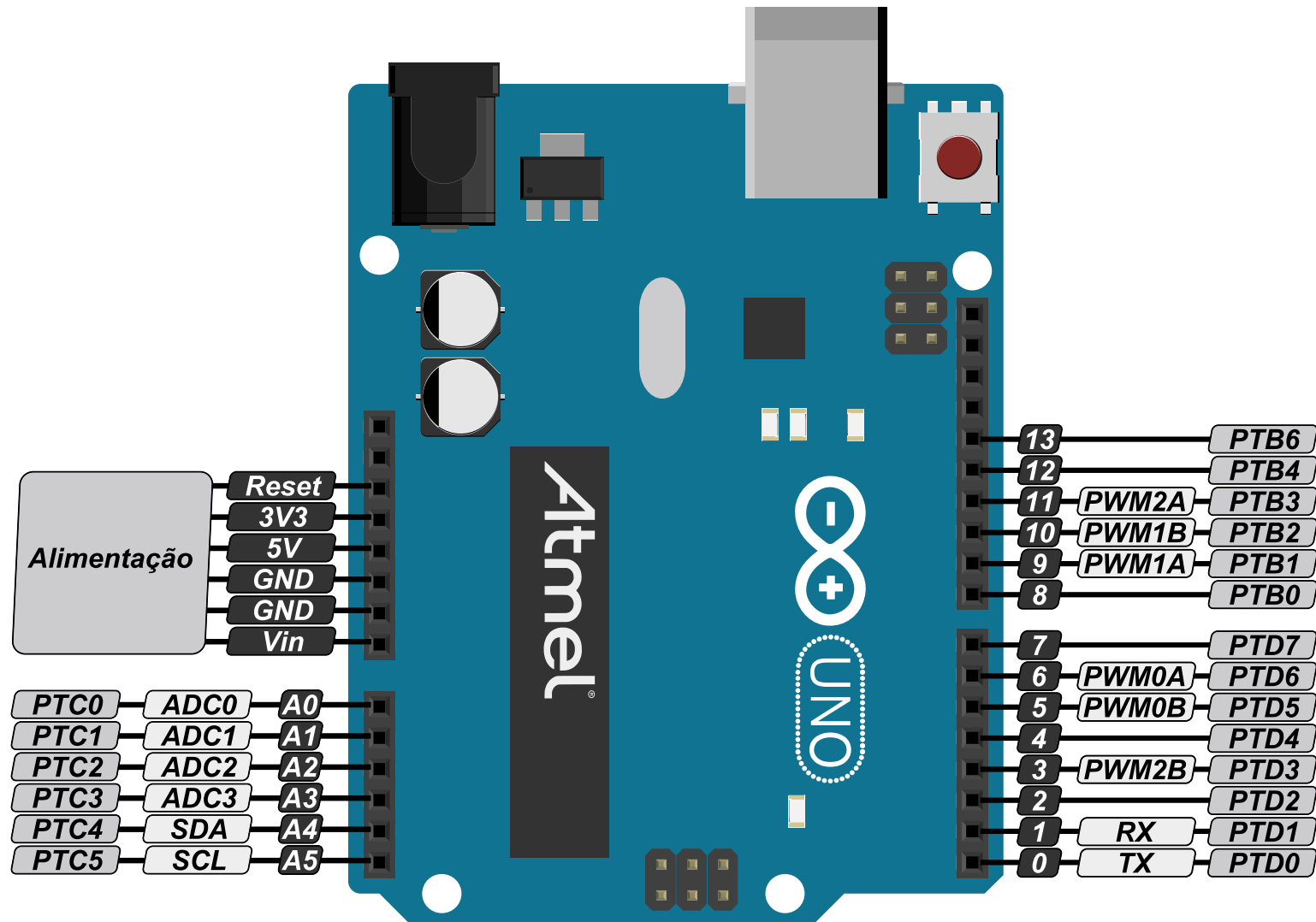
Periféricos de entrada/saída



Duas posições de memória



Arduino UNO (ATmega328P)



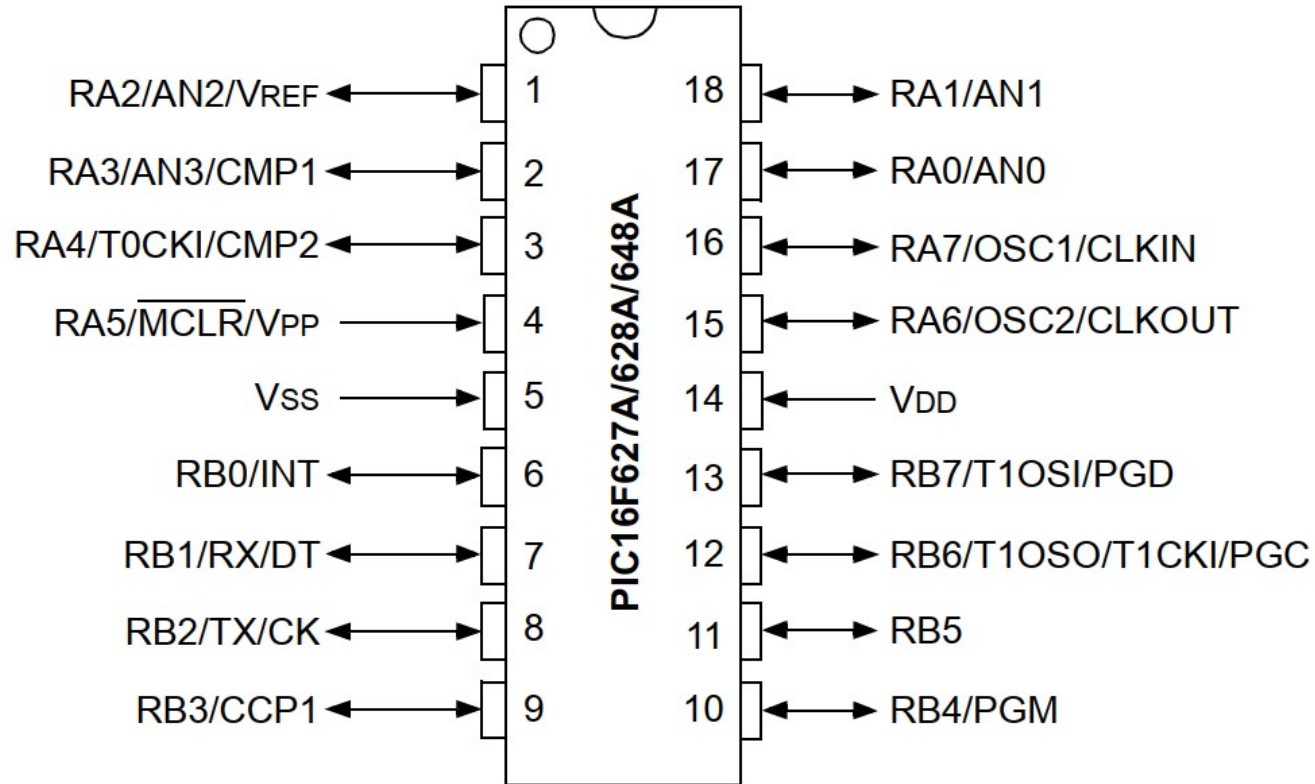
PORTAS I/O
(PORTD)

Arduino UNO (ATmega328P)

0x10 (0x30)	Reserved	-	-	-	-	-	-	-	-
0x0F (0x2F)	Reserved	-	-	-	-	-	-	-	-
0x0E (0x2E)	Reserved	-	-	-	-	-	-	-	-
0x0D (0x2D)	Reserved	-	-	-	-	-	-	-	-
0x0C (0x2C)	Reserved	-	-	-	-	-	-	-	-
0x0B (0x2B)	PORTD	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0
0x0A (0x2A)	DDRD	DDD7	DDD6	DDD5	DDD4	DDD3	DDD2	DDD1	DDD0
0x09 (0x29)	PIND	PIND7	PIND6	PIND5	PIND4	PIND3	PIND2	PIND1	PIND0
0x08 (0x28)	PORTC	-	PORTC6	PORTC5	PORTC4	PORTC3	PORTC2	PORTC1	PORTC0
0x07 (0x27)	DDRC	-	DDC6	DDC5	DDC4	DDC3	DDC2	DDC1	DDC0
0x06 (0x26)	PINC	-	PINC6	PINC5	PINC4	PINC3	PINC2	PINC1	PINC0
0x05 (0x25)	PORTB	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0

PORTD - Estado da porta (Informação)
DDRD – Direção (Entrada ou Saída)

PIC16F628A



I/O Ports

PORTA

PORTB

PIC16F628A

SUMMARY OF REGISTERS ASSOCIATED WITH PORTA

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR	Value on All Other Resets
05h	PORTA	RA7	RA6	RA5 ⁽¹⁾	RA4	RA3	RA2	RA1	RA0	xxxx 0000	qqqu 0000
85h	TRISA	TRISA7	TRISA6	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	1111 1111	1111 1111
1Fh	CMCON	C2OUT	C1OUT	C2INV	C1INV	CIS	CM2	CM1	CM0	0000 0000	0000 0000
9Fh	VRCON	VREN	VROE	VRR	—	VR3	VR2	VR1	VR0	000- 0000	000- 0000

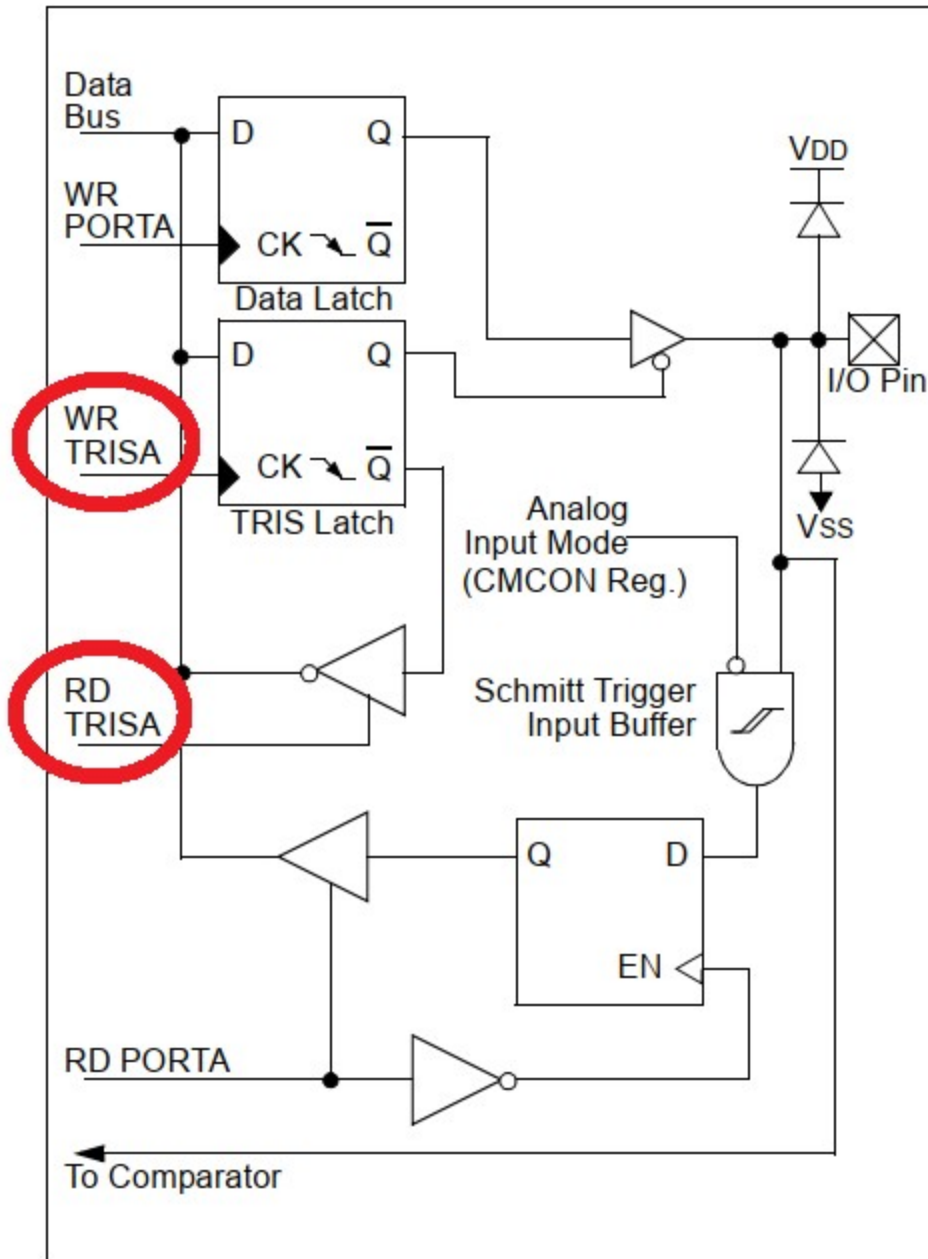
↑
Endereço

↑
Nome

BITS

PORTA - Estado da porta (Informação)
TRISA – Direção (Entrada ou Saída)

BLOCK DIAGRAM OF RA0/AN0:RA1/AN1 PINS



PIC16F628A

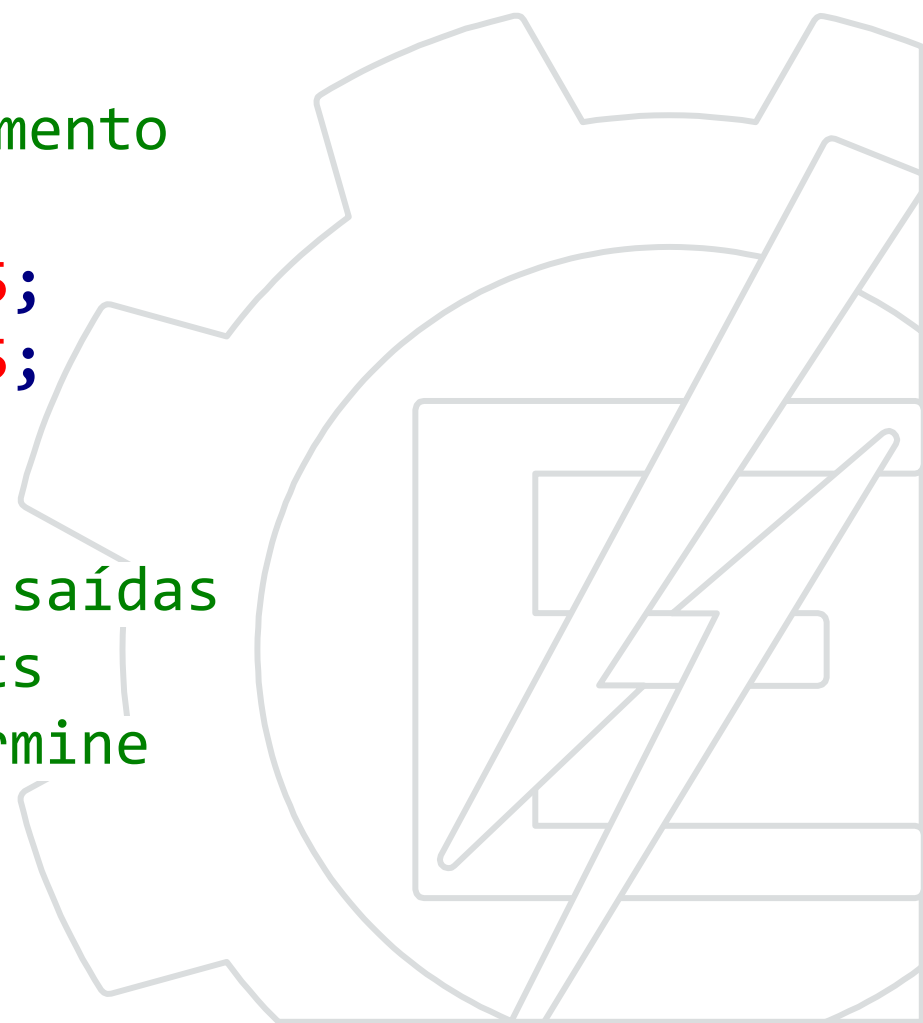
A '1' in the TRISA register puts the corresponding output driver in a High-impedance mode. A '0' in the TRISA register puts the contents of the output latch on the selected pin(s).

PINO

TRISA=1 – Entrada
TRISA=0 – Saída

Exemplo de utilização das portas

```
void main(void) {  
  
    //(unsigned char) <- 8 bits  
    //(volatile) <- podem mudar a qualquer momento  
  
    volatile unsigned char *PORTA = 0x05;  
    volatile unsigned char *TRISA = 0x85;  
  
    // TRISA 0 = saída  1 = entrada  
    *TRISA = 0; // Definindo pinos como saídas  
    *PORTA = 0xF0; // Ligando quatro bits  
    for(;;); // Evita que o programa termine  
}
```



pic16f628a.h (Biblioteca padrão)

```
// Register: PORTA
#define PORTA PORTA
extern volatile unsigned char          PORTA          __at(0x005);
#ifndef _LIB_BUILD
asm("PORTA equ 05h");
#endif
// bitfield definitions
typedef union {
    struct {
        unsigned RA0      :1;
        unsigned RA1      :1;
        unsigned RA2      :1;
        unsigned RA3      :1;
        unsigned RA4      :1;
        unsigned RA5      :1;
        unsigned RA6      :1;
        unsigned RA7      :1;
    };
} PORTAbits_t;
```

Usando #define para simplificar o uso da porta

FA5h	__ ⁽²⁾	F85h	__ ⁽²⁾
FA4h	__ ⁽²⁾	F84h	PORTE ⁽³⁾
FA3h	__ ⁽²⁾	F83h	PORTD ⁽³⁾
FA2h	IPR2	F82h	PORTC
FA1h	PIR2	F81h	PORTB
FA0h	PIE2	F80h	PORTA

Exemplo de uso

```
PORTA = 0x3A;
```

```
#define PORTA (*(volatile unsigned char*)0xF80)
```

Ponteiro para
unsigned char

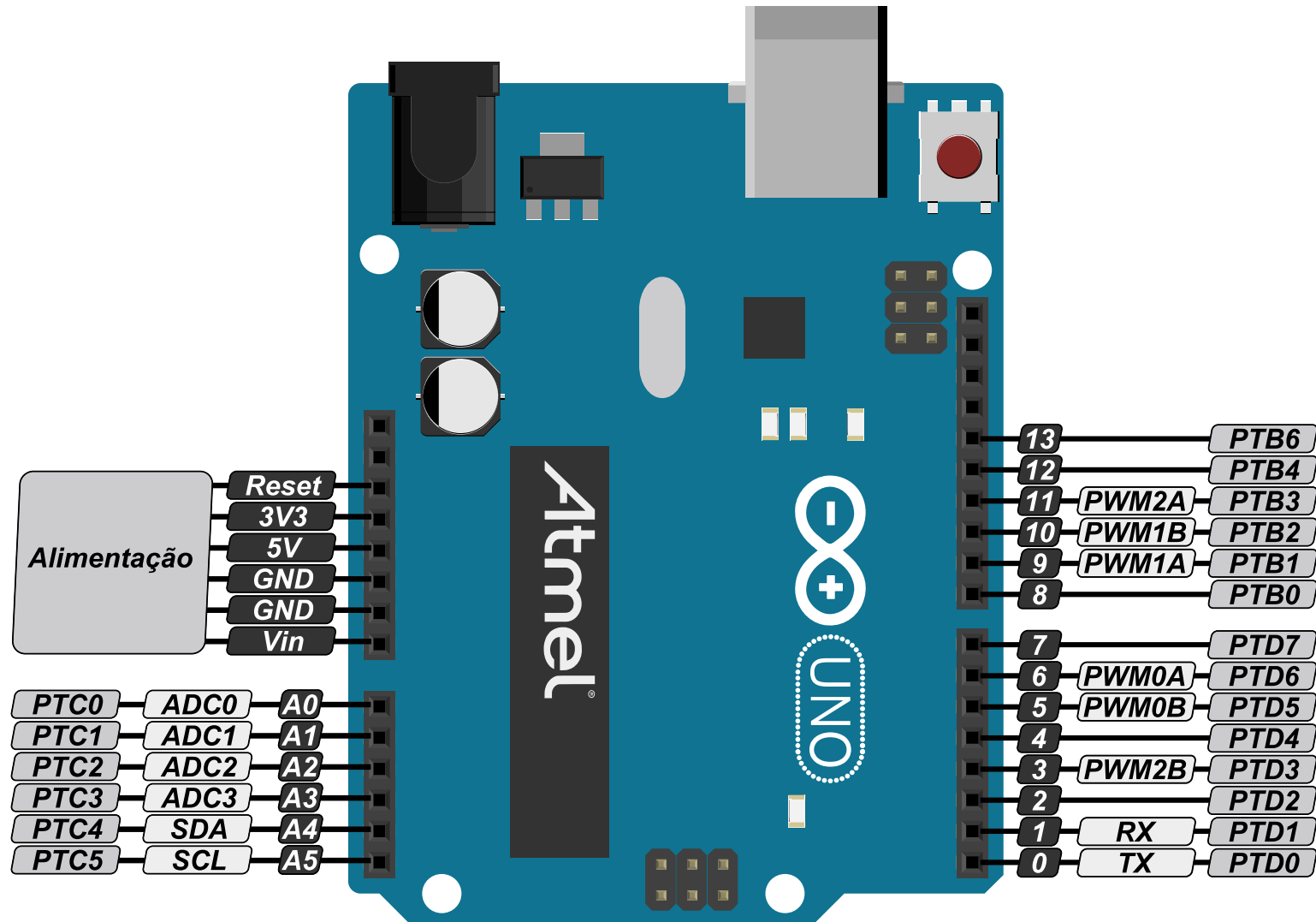
O conteúdo no endereço
apontado pelo ponteiro

Endereço

Considerações importantes

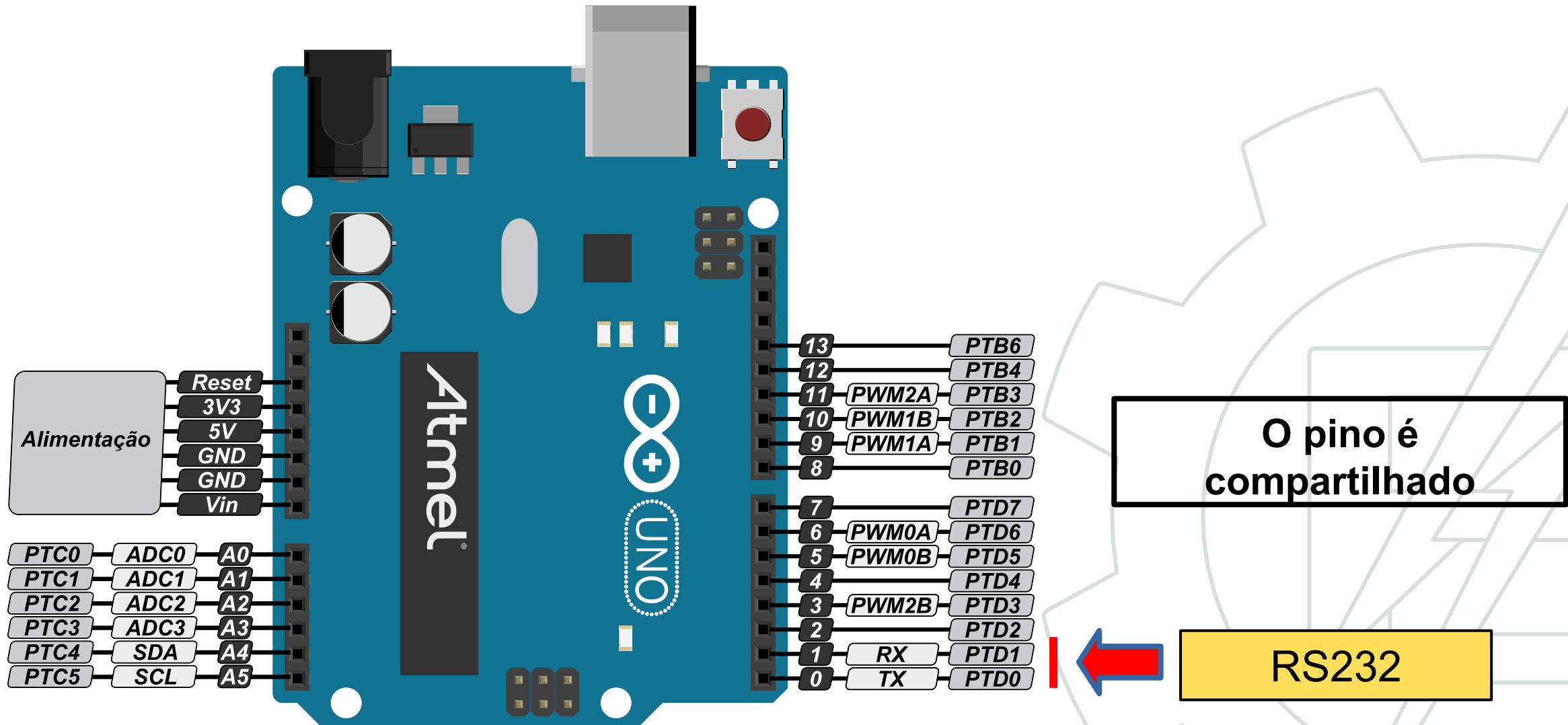


O pinos I/O das portas são compartilhados



PORTAS I/O
(PORTD)

A portas de comunicação usam os mesmos pinos utilizados pela I/O



Algumas portas podem não ser bidirecionais

PIC16F628A

Consulte sempre o datasheet

Name	Bit #	Buffer Type	Function
RA0/AN0	bit0	ST	Bi-directional I/O port/comparator input
RA1/AN1	bit1	ST	Bi-directional I/O port/comparator input
RA2/AN2/VREF	bit2	ST	Bi-directional I/O port/analog/comparator input or VREF output
RA3/AN3	bit3	ST	Bi-directional I/O port/analog/comparator input/comparator output
RA4/T0CKI	bit4	ST	Bi-directional I/O port/external clock input for TMR0 or comparator output. Output is open drain type.
RA5/MCLR/THV	bit5	ST	Input port/master clear (reset input/programming voltage input. When configured as MCLR, this pin is an active low reset to the device. Voltage on MCLR/THV must not exceed VDD during normal device operation.
RA6/OSC2/CLK-OUT	bit6	ST	Bi-directional I/O port/Oscillator crystal output. Connects to crystal or resonator in crystal oscillator mode. In ER mode, OSC2 pin outputs CLKOUT which has 1/4 the frequency of OSC1, and denotes the instruction cycle rate.
RA7/OSC1/CLKIN	bit7	ST	Bi-directional I/O port/oscillator crystal input/external clock source input.

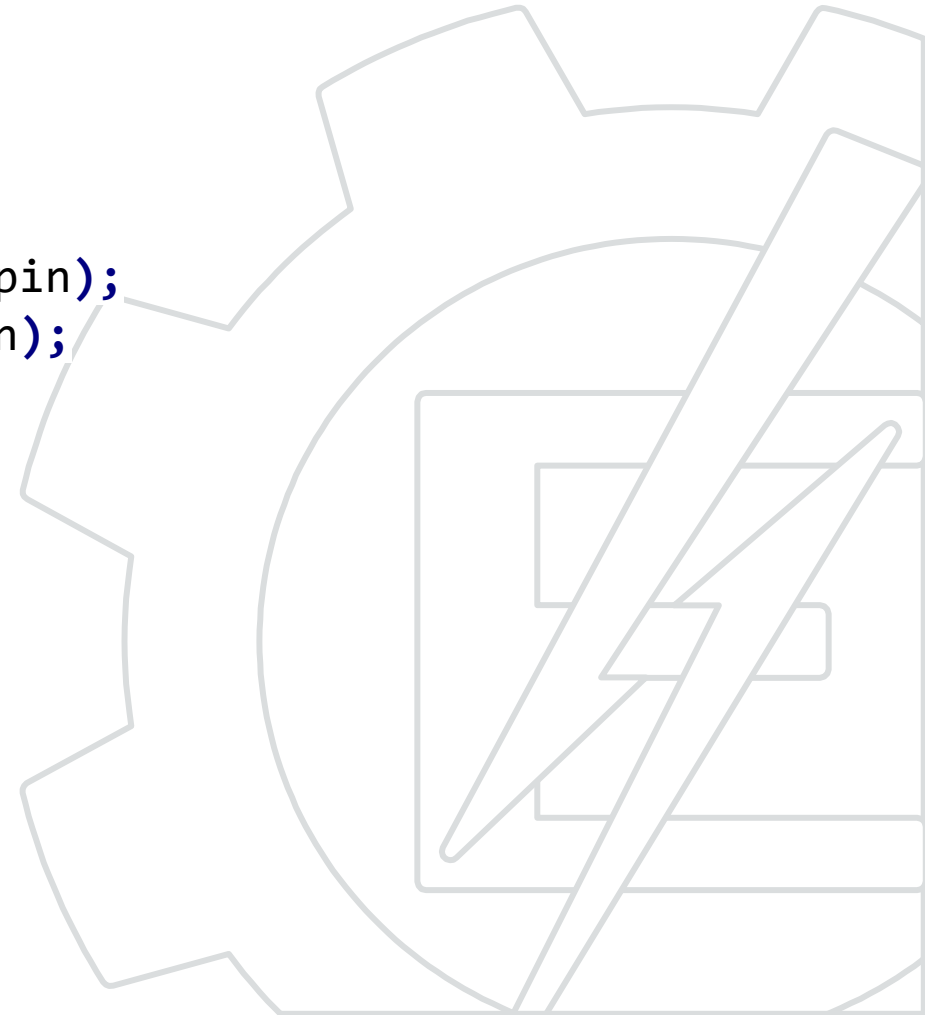
Exemplo de uma biblioteca I/O



A biblioteca do Arduino

- Configuração como entrada/saída
 - `pinMode(pin, mode);`
- Implementação do Arduino:

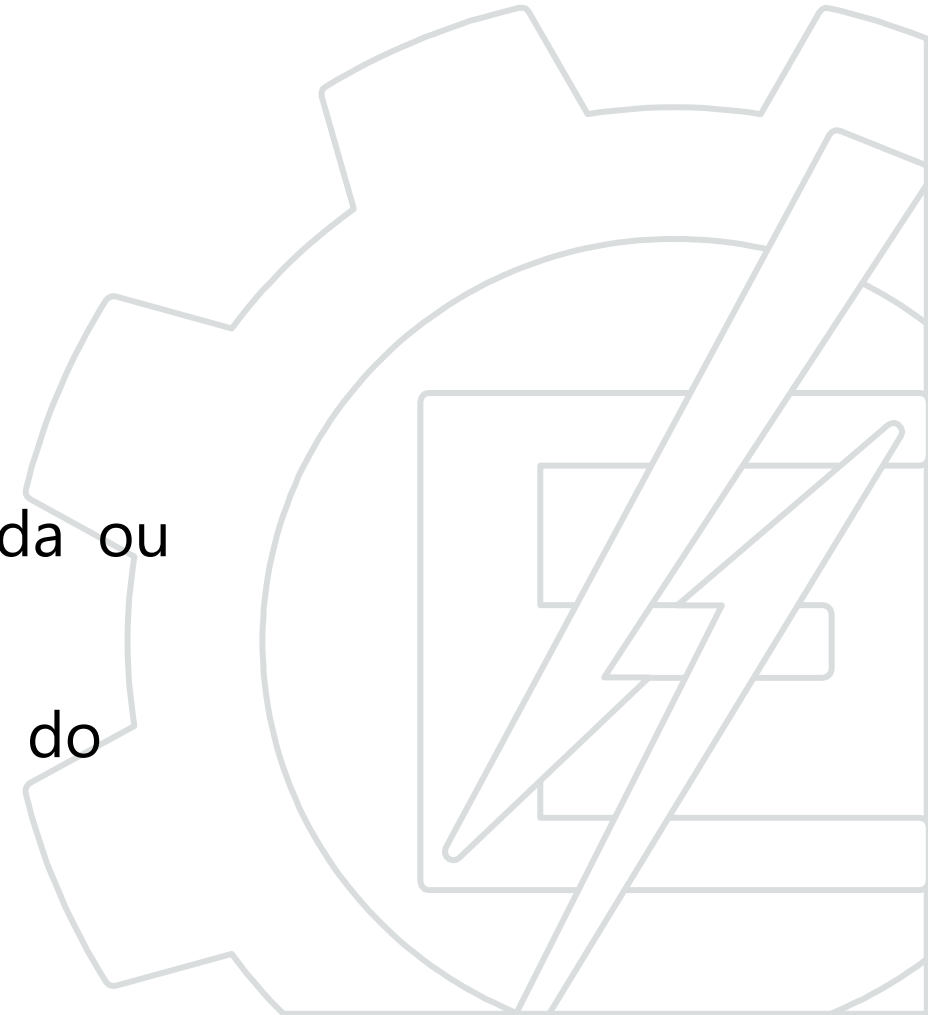
```
void pinMode(uint8_t pin, uint8_t mode){
    uint8_t bit = digitalPinToBitMask(pin);
    uint8_t port = digitalPinToPort(pin);
    volatile uint8_t *reg, *out;
    reg = portModeRegister(port);
    out = portOutputRegister(port);
    if (mode == INPUT) {
        *reg &= ~bit;
        *out &= ~bit;
    } else if (mode == OUTPUT){
        *reg |= bit;
    }
}
```



Biblioteca do Arduino

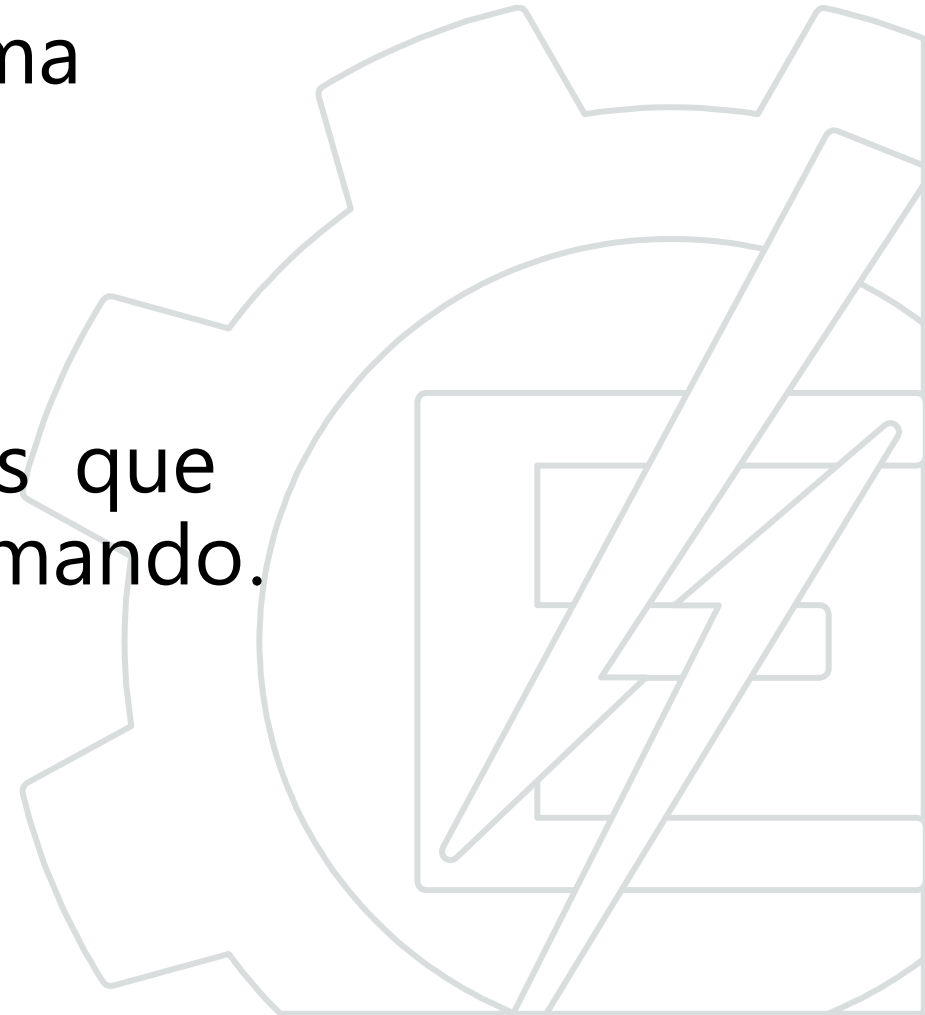
Funções de acesso:

- `digitalWrite()` : liga/desliga um terminal
- `digitalRead()`: lê o estado de um terminal
- Funções de configuração:
 - `pinMode()`: define se o terminal será entrada ou saída
 - `systemInit()`: inicializa os periféricos básicos do sistema



Exemplo de criação de uma biblioteca

- Podemos criar para o PIC18f4520 uma biblioteca idêntica a da plataforma Arduino.
- Essa biblioteca pode utilizar `#defines` que facilitam a identificação de cada comando.



Biblioteca IO

Arquivo: io.h

```
#ifndef IO_H
#define IO_H

#define OUTPUT 0
#define INPUT 1
#define LOW 0
#define HIGH 1
```

```
enum pin_label{
    PIN_A0,PIN_A1,PIN_A2,PIN_A3,PIN_A4,PIN_A5,PIN_A6,PIN_A7,
    PIN_B0,PIN_B1,PIN_B2,PIN_B3,PIN_B4,PIN_B5,PIN_B6,PIN_B7,
    PIN_C0,PIN_C1,PIN_C2,PIN_C3,PIN_C4,PIN_C5,PIN_C6,PIN_C7,
    PIN_D0,PIN_D1,PIN_D2,PIN_D3,PIN_D4,PIN_D5,PIN_D6,PIN_D7,
    PIN_E0,PIN_E1,PIN_E2,PIN_E3,PIN_E4,PIN_E5,PIN_E6,PIN_E7
};
```

```
void digitalWrite(int pin, int value);
int digitalRead(int pin);
void pinMode(int pin, int type);
#endif
```

No arquivo header (.h)
colocar somente protótipos
de funções e definições

Biblioteca IO

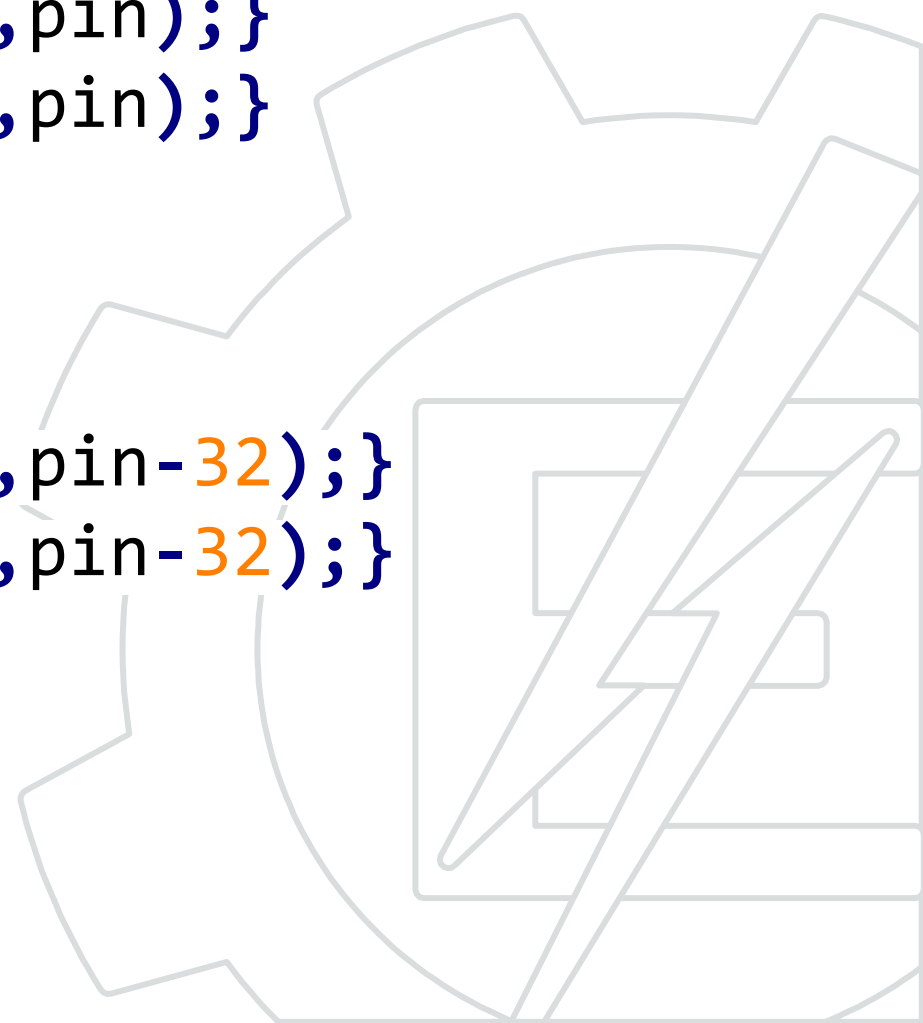
Arquivo: io.c

No arquivo code (.c) definir o funcionamento das funções

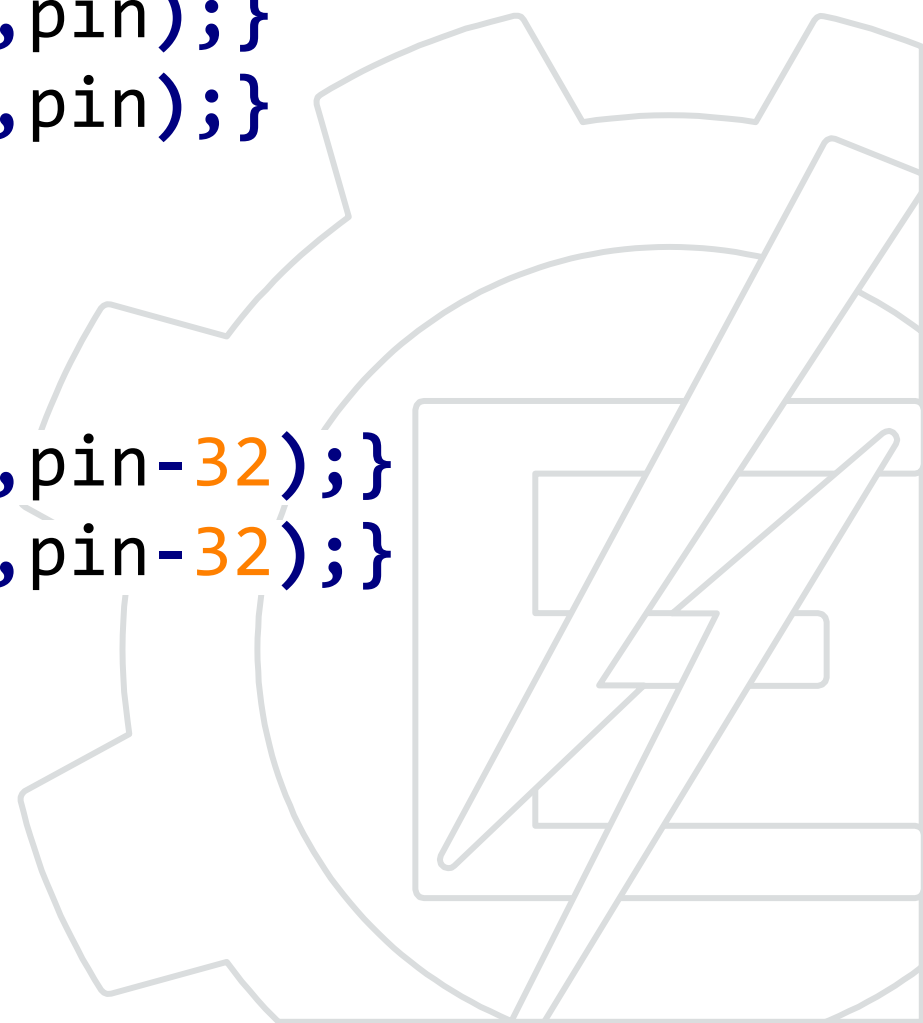
```
int digitalRead (int pin){  
    if(pin < 8){  
        return bitTst(PORTA, pin);  
    }  
  
    //outras portas...  
    else if(pin < 40){  
        return bitTst(PORTE, pin - 32);  
    }  
    return -1;  
}
```



```
void digitalWrite(int pin, int value){
    if(pin <8){
        if (value){ bitSet(PORTA,pin);}
        else{       bitClr(PORTA,pin);}
    }
    //outras portas...
    else if(pin<40){
        if (value){ bitSet(PORTE,pin-32);}
        else{       bitClr(PORTE,pin-32);}
    }
}
```



```
void pinMode      (int pin, int type) {  
    if(pin <8){  
        if (type) { bitSet(TRISA,pin);}   
        else{      bitClr(TRISA,pin);}   
    }  
    //outras portas...  
    else if(pin<40){  
        if (type) { bitSet(TRISE,pin-32);}   
        else{      bitClr(TRISE,pin-32);}   
    }  
}
```




```
#ifndef IO_H  
#define IO_H
```

```
#define OUTPUT 0  
#define INPUT 1  
#define LOW 0  
#define HIGH 1
```

```
enum pin_label{  
    PIN_A0,PIN_A1,PIN_A2,PIN_A3,PIN_A4,PIN_A5,PIN_A6,PIN_A7,  
    PIN_B0,PIN_B1,PIN_B2,PIN_B3,PIN_B4,PIN_B5,PIN_B6,PIN_B7,  
    PIN_C0,PIN_C1,PIN_C2,PIN_C3,PIN_C4,PIN_C5,PIN_C6,PIN_C7,  
    PIN_D0,PIN_D1,PIN_D2,PIN_D3,PIN_D4,PIN_D5,PIN_D6,PIN_D7,  
    PIN_E0,PIN_E1,PIN_E2,PIN_E3,PIN_E4,PIN_E5,PIN_E6,PIN_E7  
};
```

```
void digitalWrite(int pin, int value);  
int digitalRead(int pin);  
void pinMode(int pin, int type);  
#endif
```

Biblioteca IO
Arquivo: io.h

Cuidado

**Usando funções o seu programa
fica maior e mais lento**



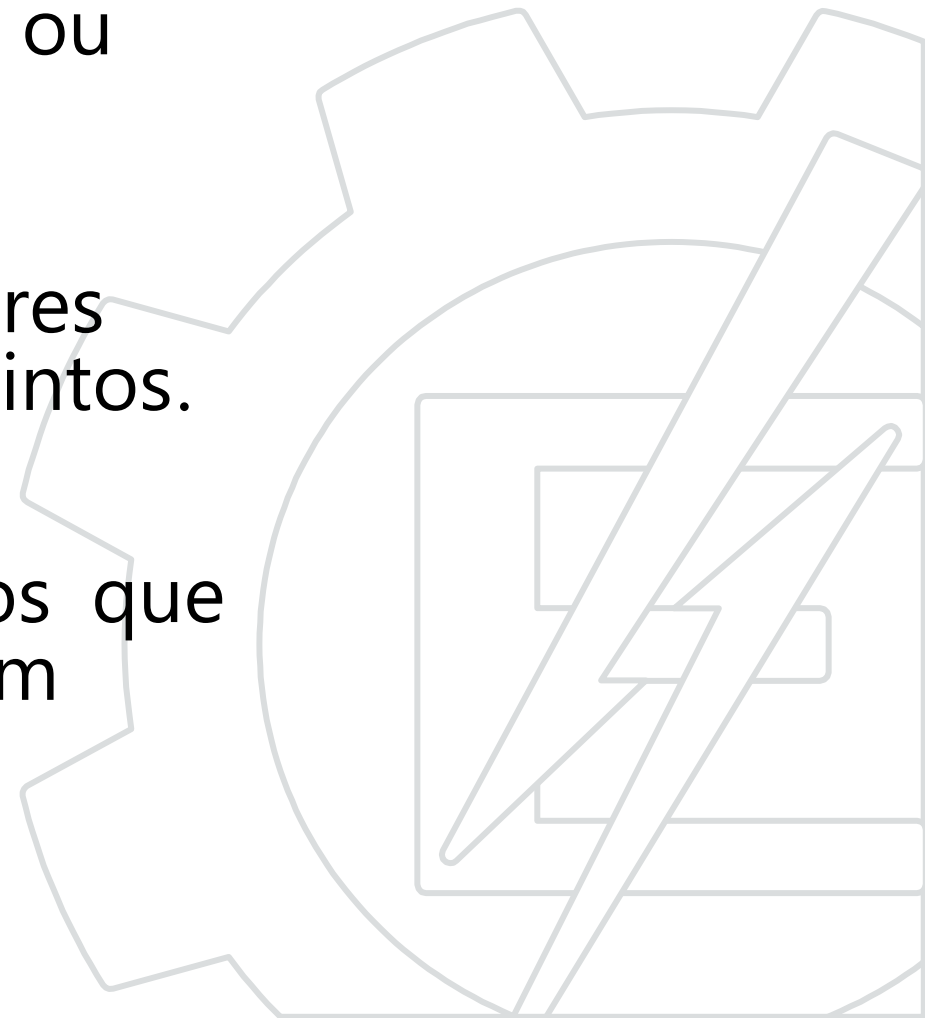
Imagem: www.graphicsfactory.com

Saídas Digitais



Saídas Digitais

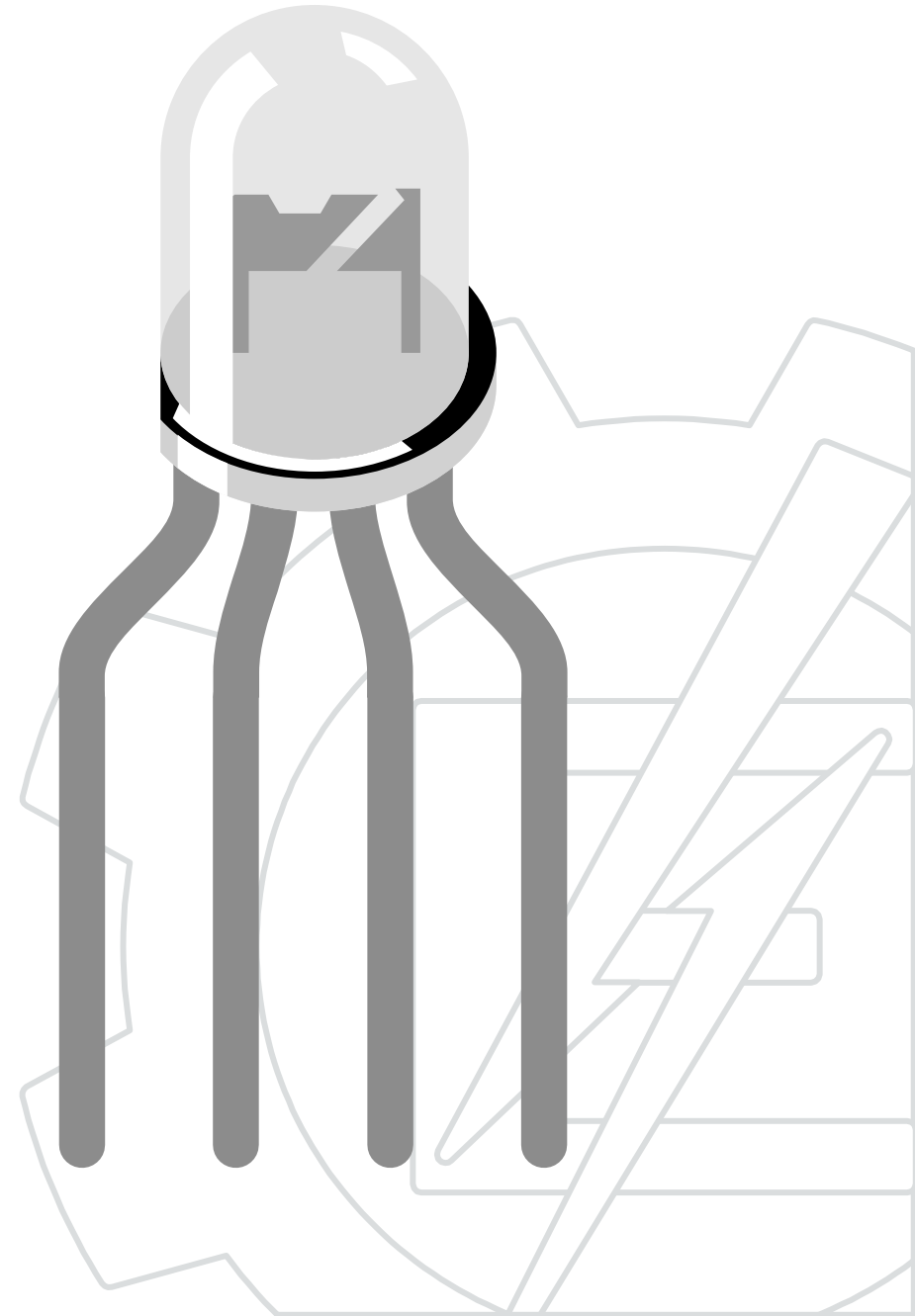
- Diversos componentes eletrônicos possuem apenas dois tipos de estados: ligados ou desligados.
- As saídas digitais dos microcontroladores apresentam somente dois estados distintos.
- Podemos utilizar diretamente comandos que ajustam as portas ou criar funções com `bitClr()` e `BitSet()`.



Saídas Digitais

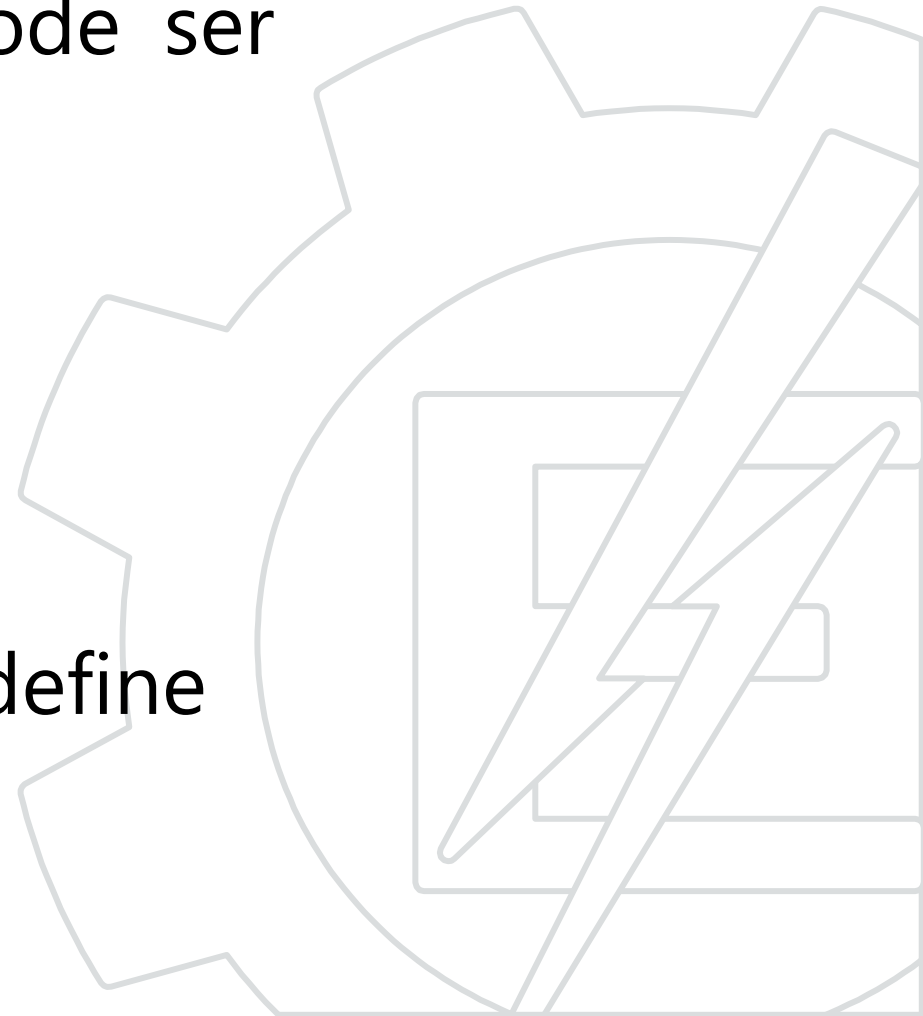
Exemplo de biblioteca

- Um led RGB é dispositivo composto de três leds:
 - Vermelho (R)
 - Verde (G)
 - Azul (B)



Criação da biblioteca do led RGB

- O led RGB é um dispositivo que pode ser controlado por saída digital.
- A biblioteca pode gerar várias combinações de cores.
- Cada cor será implementada por `#define`



Arquivo rgb.h

```
#ifndef RGB
#define RGB

//definição das cores
#define OFF      0
#define RED      1
#define GREEN    2
#define BLUE     4
#define YELLOW   (RED+GREEN)
#define CYAN     (GREEN+BLUE)
#define PURPLE   (RED+BLUE)
#define WHITE    (RED+GREEN+BLUE)

void rgbColor(unsigned char cor);
void rgbInit(void);

#endif
```



Uma função init() é necessário

```
void rgbInit(void) {  
    pinMode(LED_R_PIN, OUTPUT);  
    pinMode(LED_G_PIN, OUTPUT);  
    pinMode(LED_B_PIN, OUTPUT);  
}
```

Obs: LED_R_PIN, HIGH e outras são definidas em outras partes do código.

```
#include "io.h"  
#include "rgb.h"
```

Arquivo rgb.c

```
void rgbColor(unsigned char cor)  
{  
    if (cor & 1)    digitalWrite(LED_R_PIN, HIGH);  
        else digitalWrite(LED_R_PIN, LOW);  
  
    if (cor & 2) digitalWrite(LED_G_PIN, HIGH);  
        else digitalWrite(LED_G_PIN, LOW);  
  
    if (cor & 4) digitalWrite(LED_B_PIN, HIGH);  
        else digitalWrite(LED_B_PIN, LOW);  
}
```

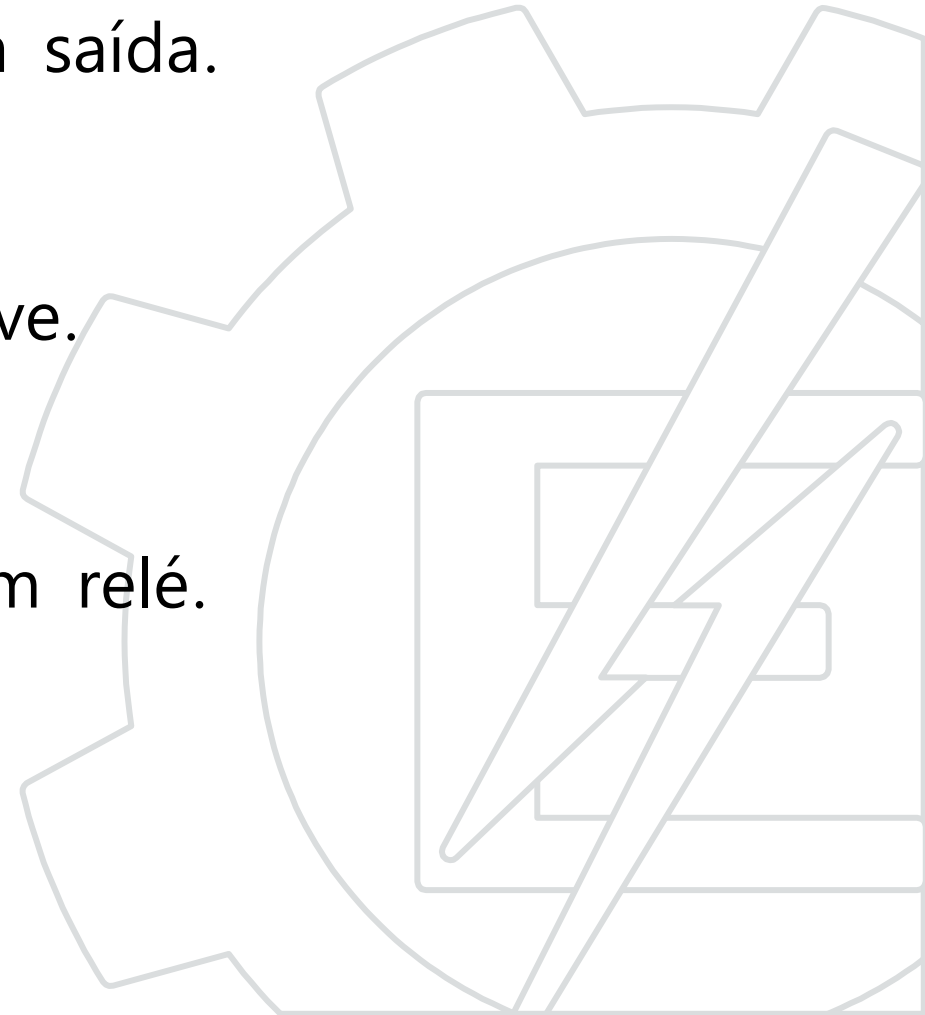
Obs: LED_R_PIN, HIGH e outras são definidas em outras partes do código.

Outros dispositivos digitais

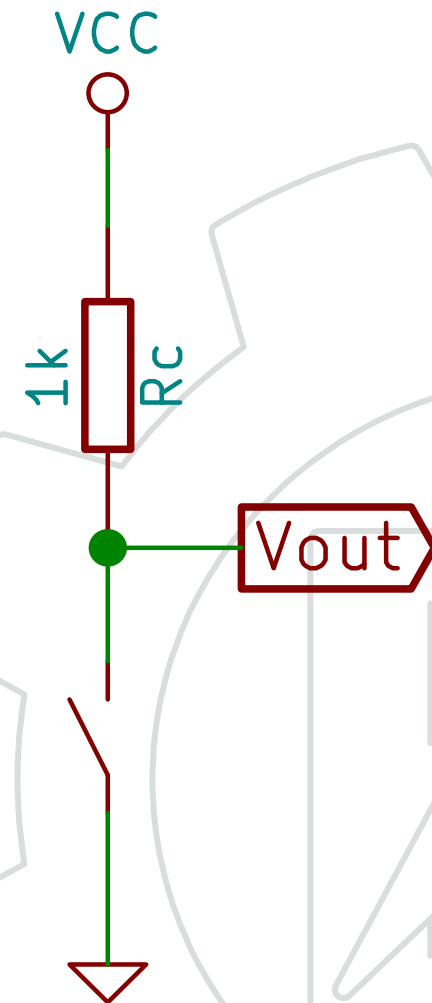
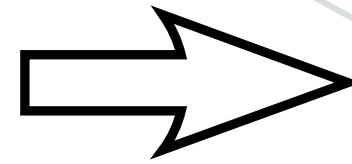
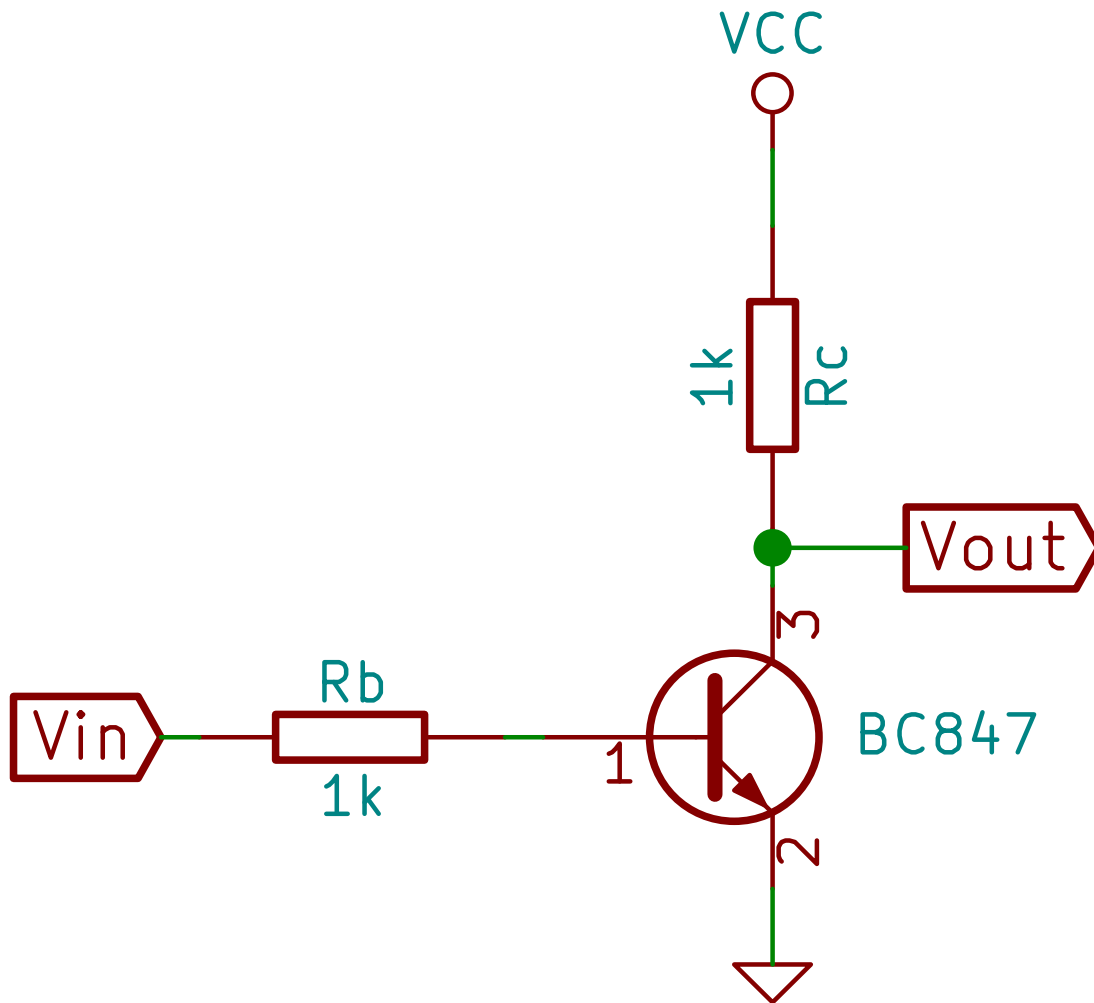


Outras saídas digitais

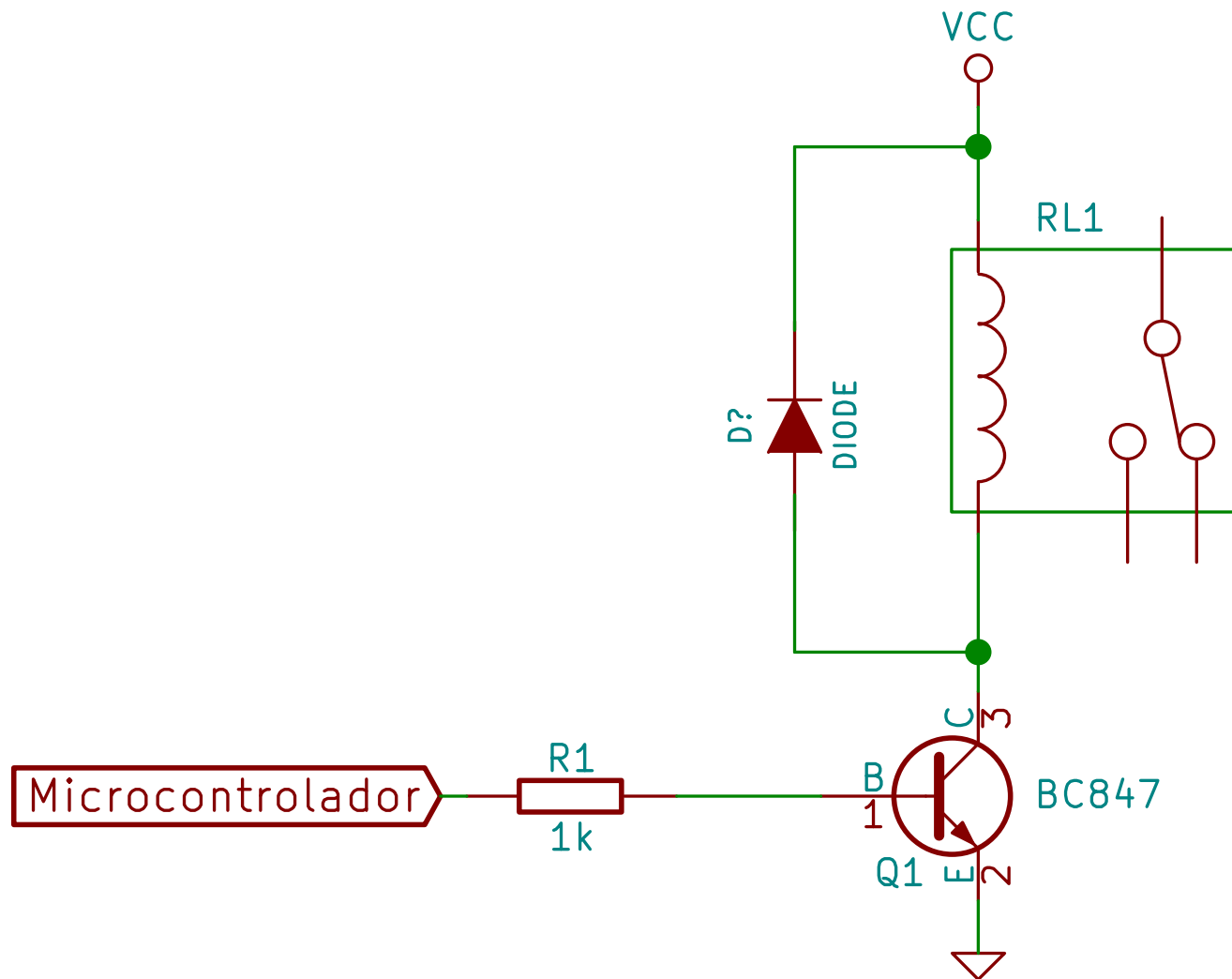
- Transistor
 - Pode funcionar como um amplificador ou chave.
 - Como chave permite ligar cargas em sua saída.
- Relé
 - Similar ao transistor operando como chave.
- Relé de estado sólido
 - Circuito eletrônico que funciona como um relé.



Transistor operando como chave



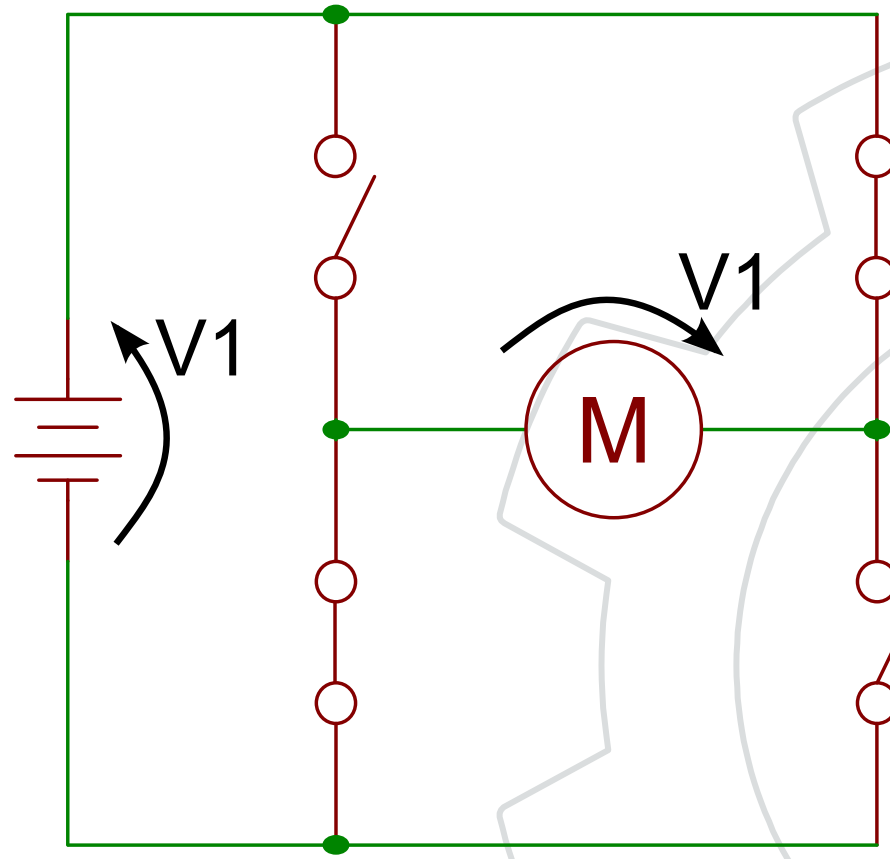
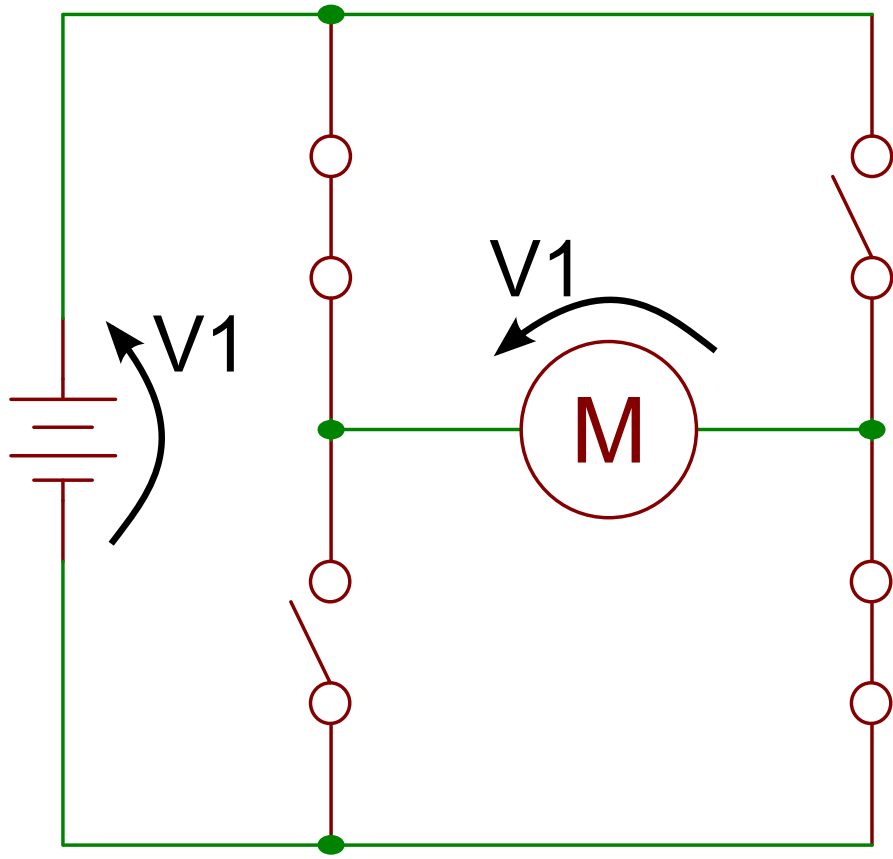
Acionamento por relé



Acionamento de uma ponte H



Exemplo de uso da ponte H



```
#define sw1A 3
#define sw1B 4
#define sw2A 5
#define sw2B 6

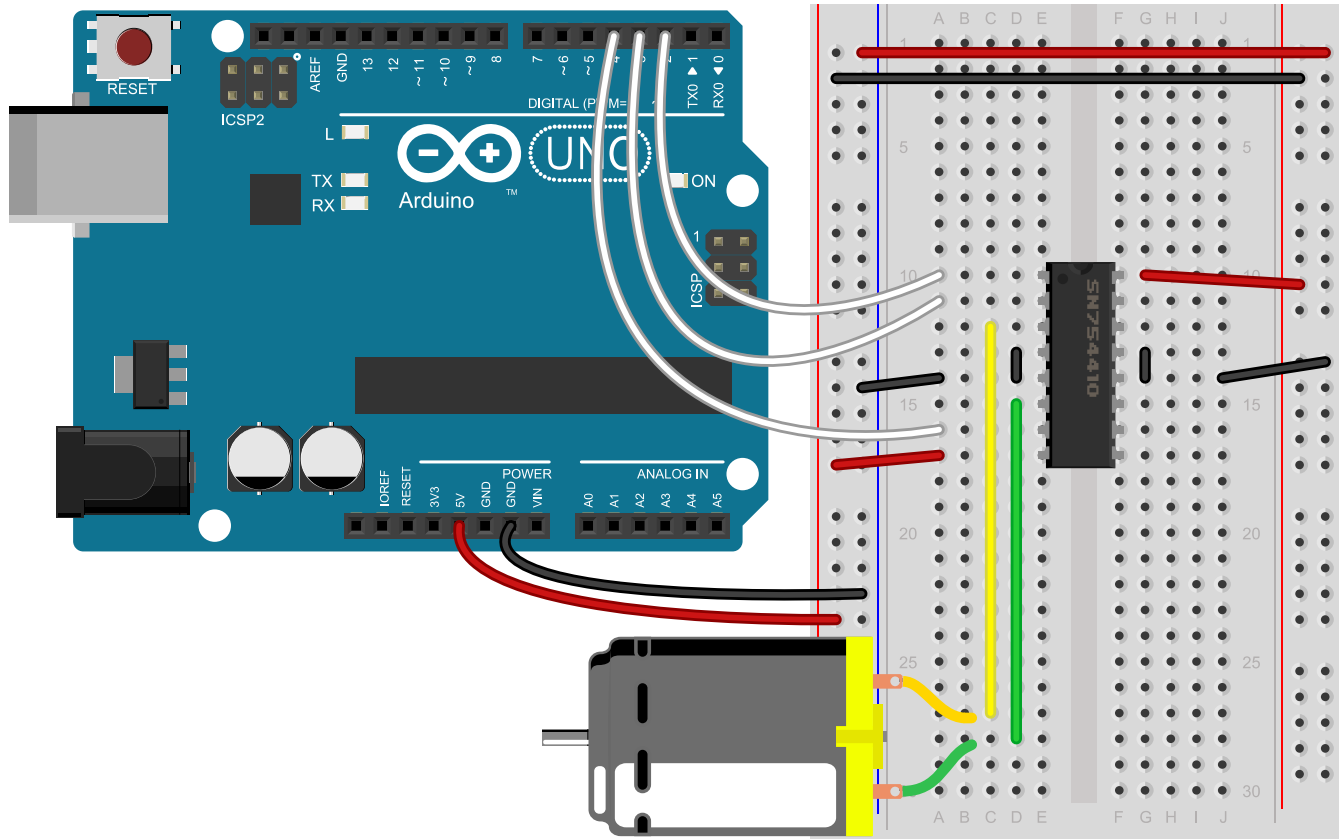
void initMotorControl(void){
    //configura 4 terminais como saída
    pinMode(sw1A, OUTPUT);
    pinMode(sw1B, OUTPUT);
    pinMode(sw2A, OUTPUT);
    pinMode(sw2B, OUTPUT);
}

void motorOff(void){
    digitalWrite(sw1A, LOW);
    digitalWrite(sw1B, LOW);
    digitalWrite(sw2A, LOW);
    digitalWrite(sw2B, LOW);
}
```

```
void motorOnLeft(void){
    //sempre desliga primeiro
    digitalWrite(sw2A, LOW);
    digitalWrite(sw2B, LOW);
    digitalWrite(sw1A, HIGH);
    digitalWrite(sw1B, HIGH);
}

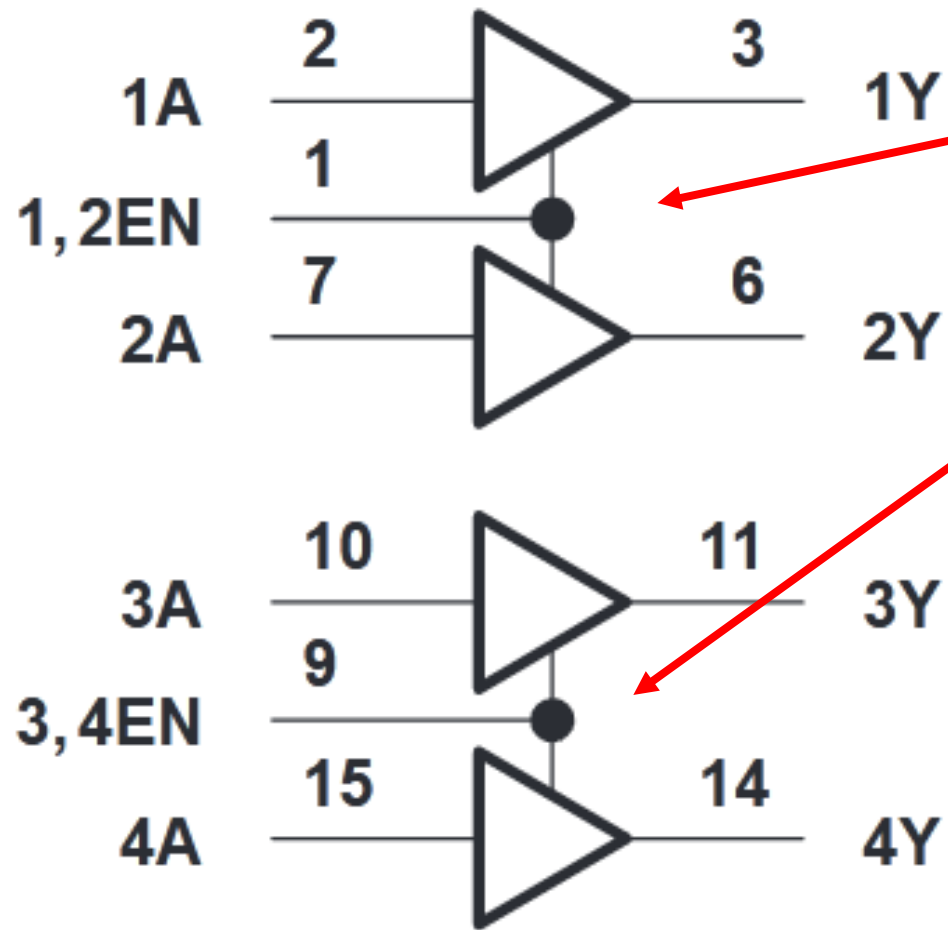
void motorOnRight(void){
    //sempre desliga primeiro
    digitalWrite(sw1A, LOW);
    digitalWrite(sw1B, LOW);
    digitalWrite(sw2A, HIGH);
    digitalWrite(sw2B, HIGH);
}
```

Ponte H com circuito dedicado

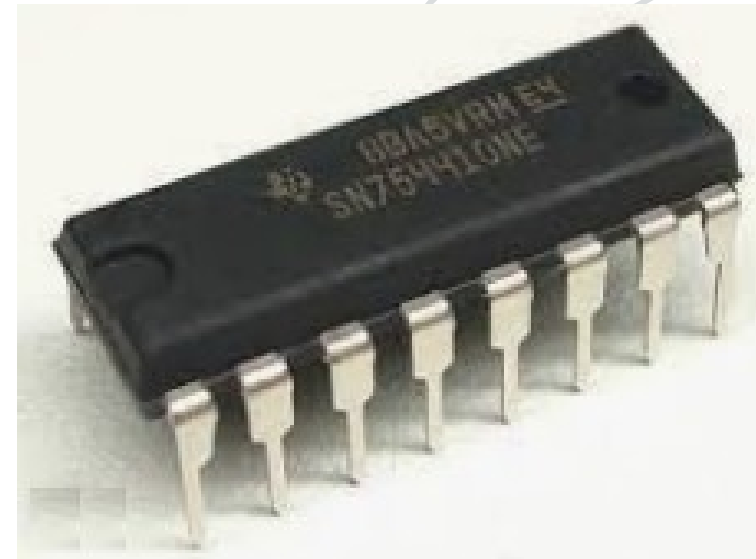


Um circuito integrado pode ser usado para controlar a direção.

SN754410



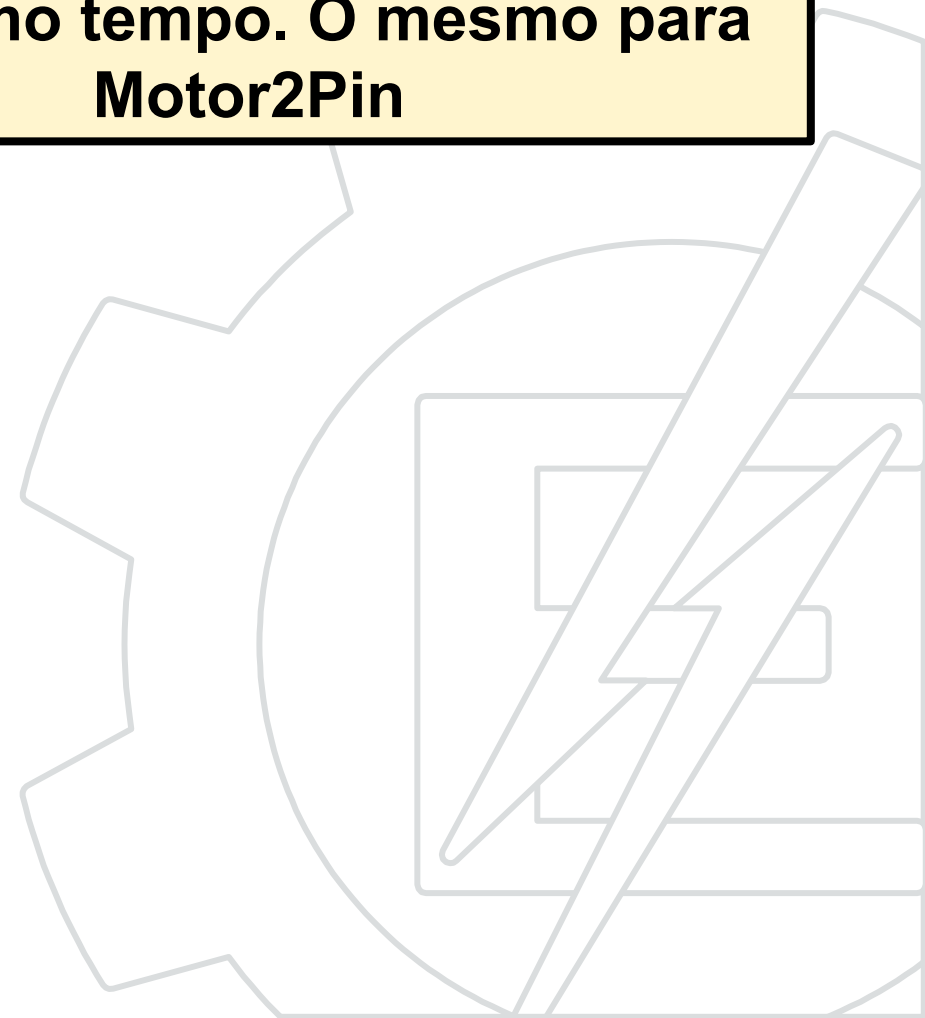
Este circuito integrado tem pinos enable



```
#define motor1Pin 3
#define motor2Pin 4
#define enablePin 2
```

```
void initMotorControl(void){
    pinMode(motor1Pin, OUTPUT);
    pinMode(motor2Pin, OUTPUT);
    pinMode(enablePin, OUTPUT);
}
void motorOff(void){
    digitalWrite(enablePin, LOW);
}
void motorOnLeft(void){
    digitalWrite(motor1Pin, LOW);
    digitalWrite(motor2Pin, HIGH);
    digitalWrite(enablePin, HIGH);
}
void motorOnRight(void){
    digitalWrite(motor2Pin, LOW);
    digitalWrite(motor1Pin, HIGH);
    digitalWrite(enablePin, HIGH);
}
```

**Motor1Pin ativa duas saídas do CI
ao mesmo tempo. O mesmo para
Motor2Pin**



Expansão de saídas digitais

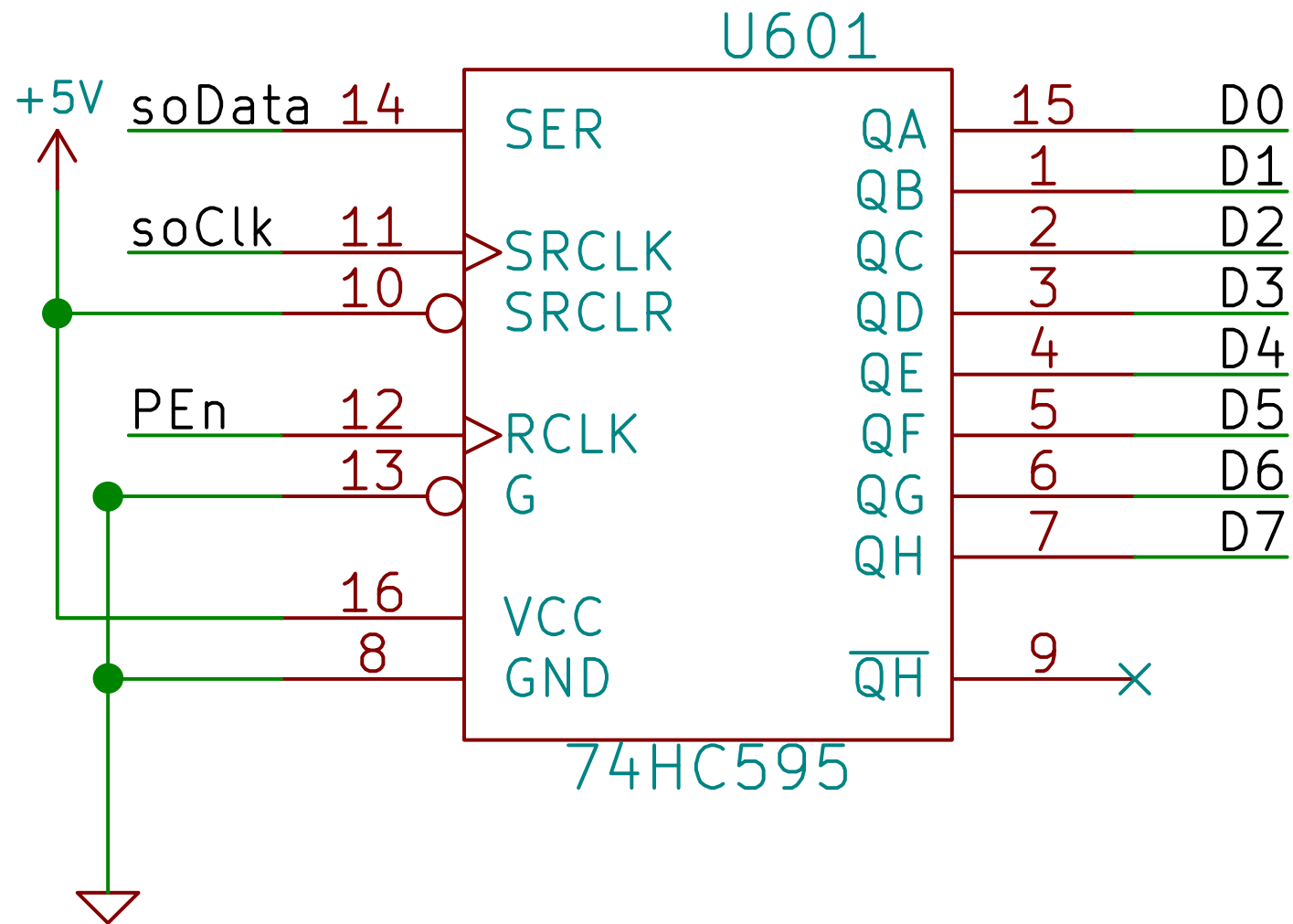


Expansão de saídas

- O custo de um microcontrolador é dependente da quantidade de saídas disponíveis.
- Se o micro não possui saídas suficientes podemos utilizar circuitos para expansão de saídas, como exemplo:
 - conversores de serial para paralelo.
 - expansores de IO.
- Toda abordagem insere algum problema.
 - custo do sistema.
 - atraso na resposta.
 - aumento da complexidade.



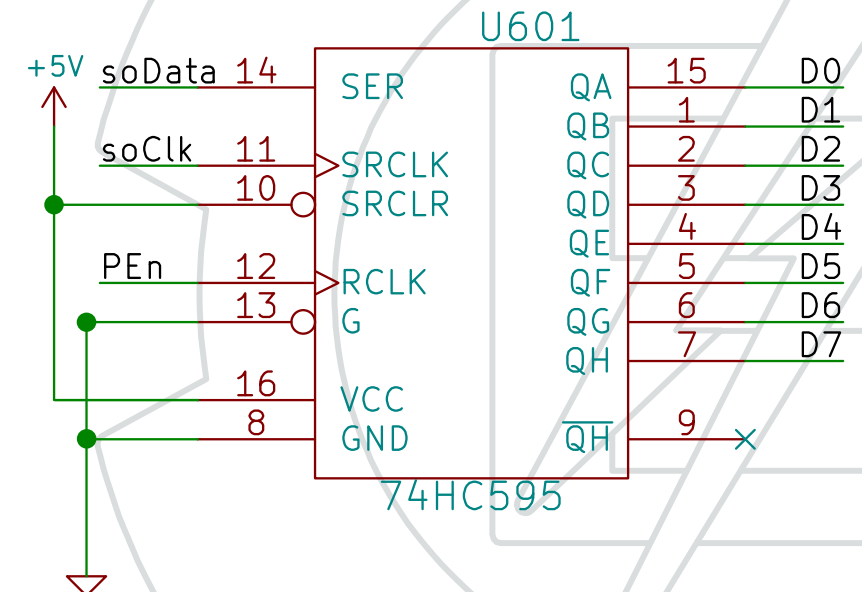
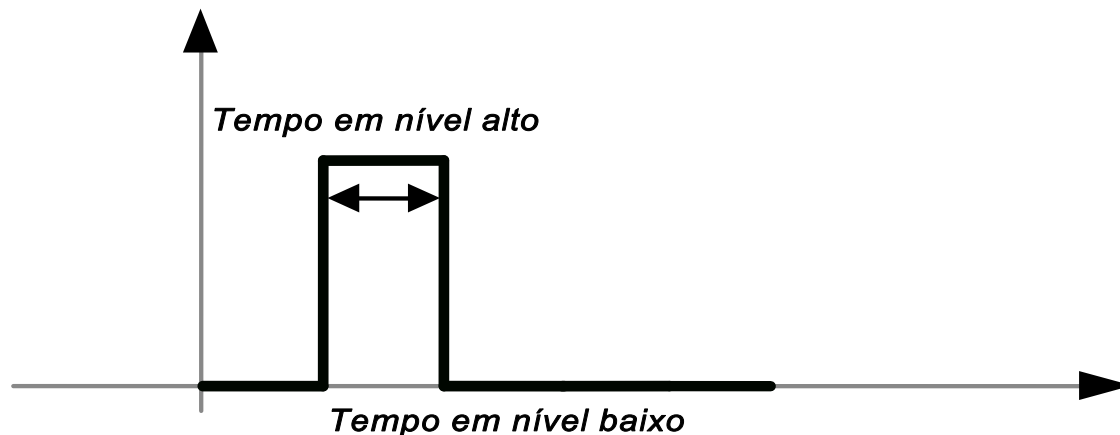
Expansão de saídas



- Entrada serial
- Saída paralela

Funcionamento do 74HC595

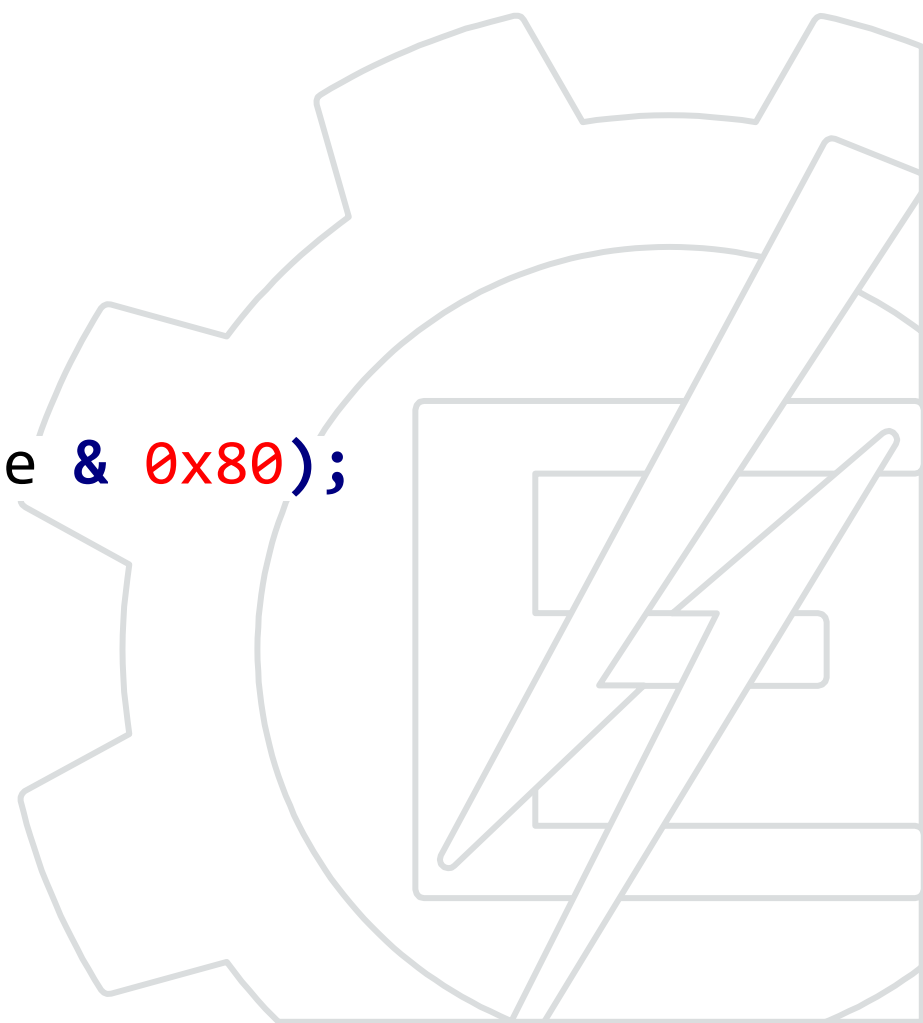
- Ajustando os bits na saída do 74HC595:
 - O primeiro bit é colocado em soData.
 - Um pulso de clock em soClk ajusta a saída QA.
 - O segundo bit é colocado em soData.
 - Um novo pulso em soClk ajusta a saída QB e assim até finalizar com a saída QH.
 - O Pino PEn liga o dispositivo.



Exemplo de código

- A função abaixo recebe uma variável com 8 bits e envia cada bit serialmente.

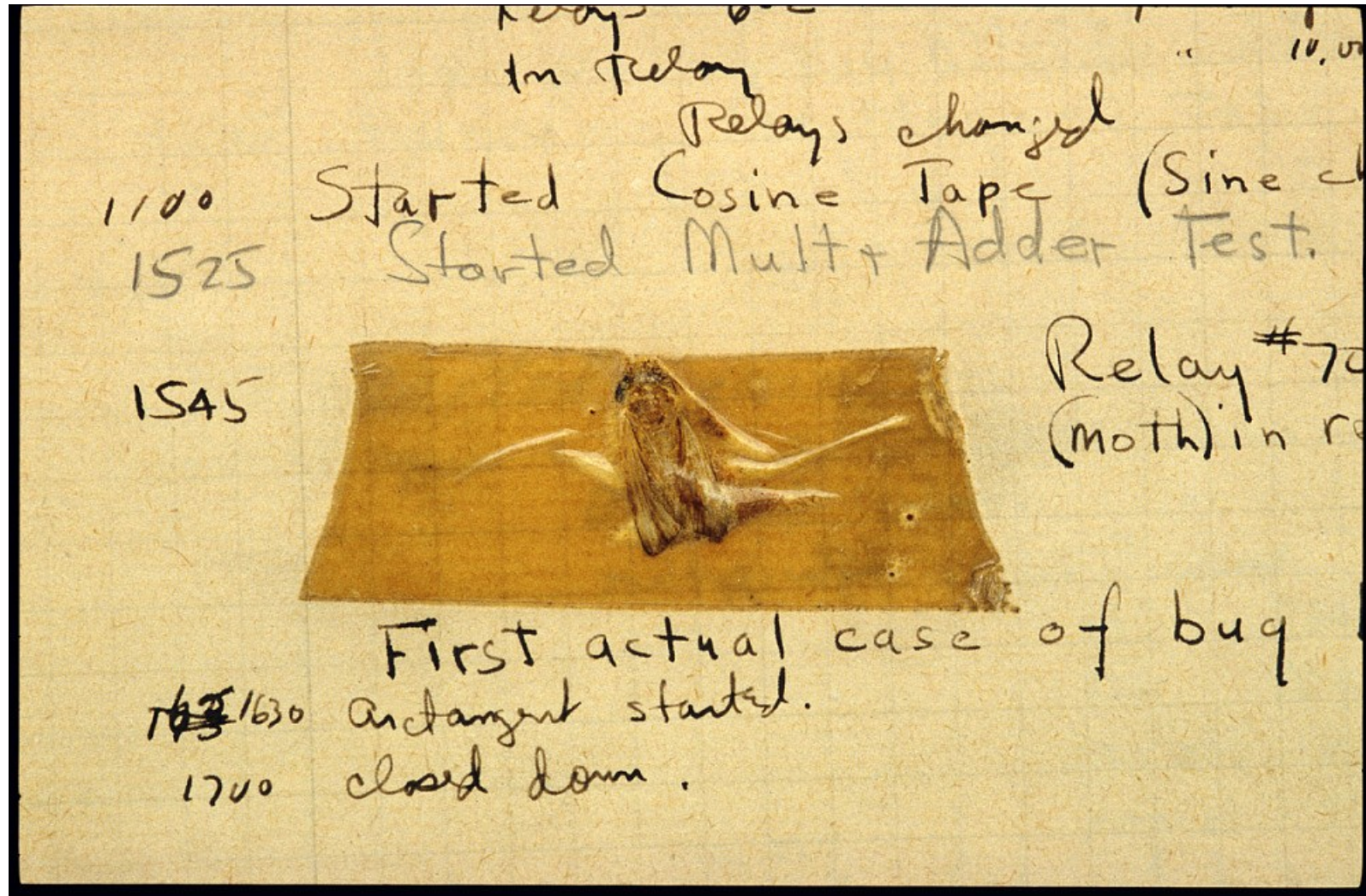
```
void soWrite(char value) {  
    unsigned char i;  
    digitalWrite(SO_CLK_PIN, LOW);  
    for (i = 0; i < 8; i++) {  
        digitalWrite(SO_DATA_PIN, value & 0x80);  
        PulseClockData();  
        value <<= 1;  
    }  
    PulseEnClock();  
}
```



BUGS!

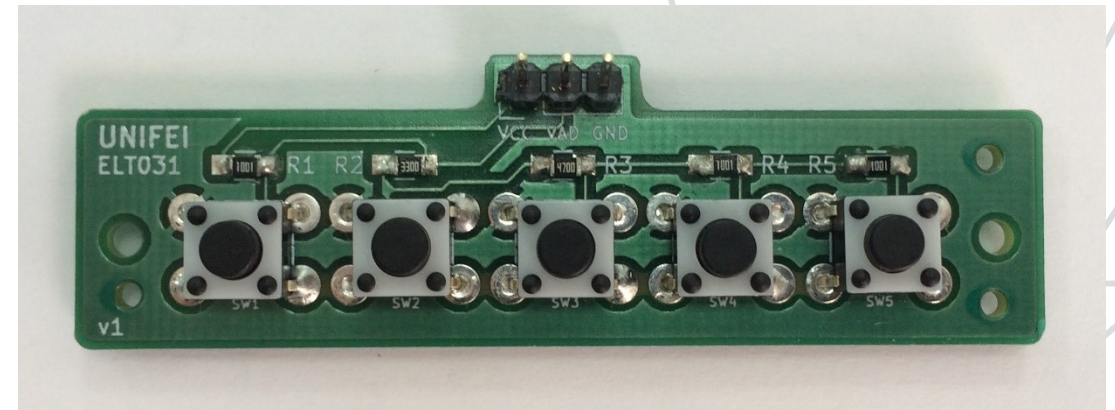
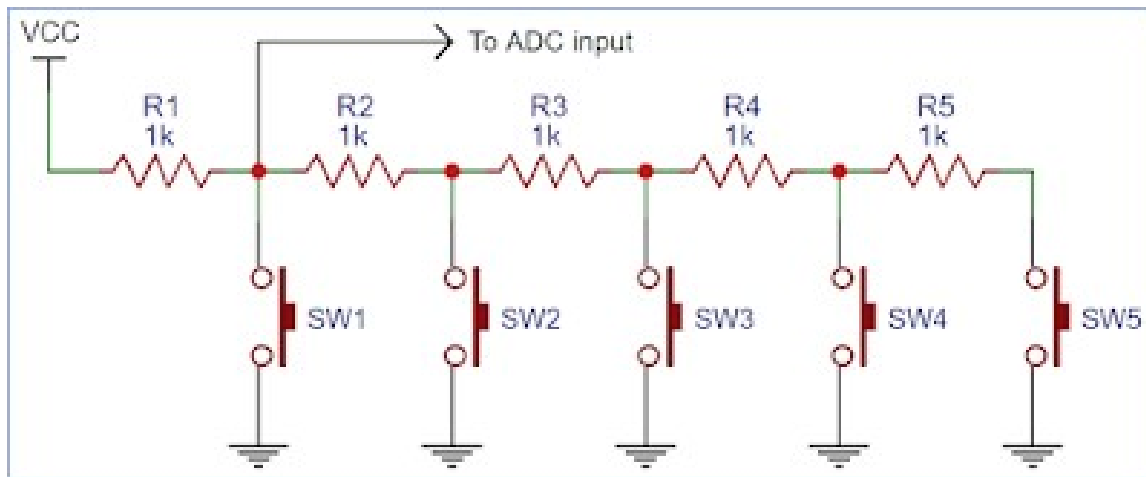


Primeiro bug



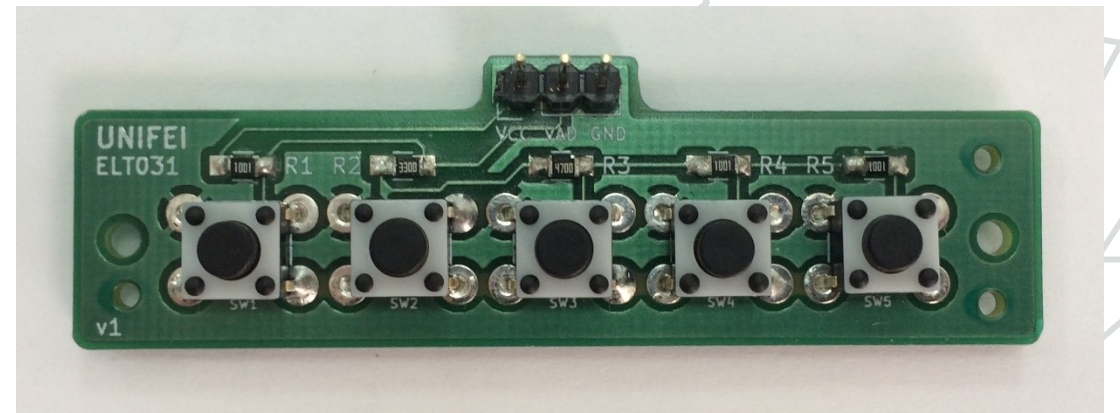
Um exemplo de BUG

Teclado com divisores de tensão conectado a uma entrada analógica



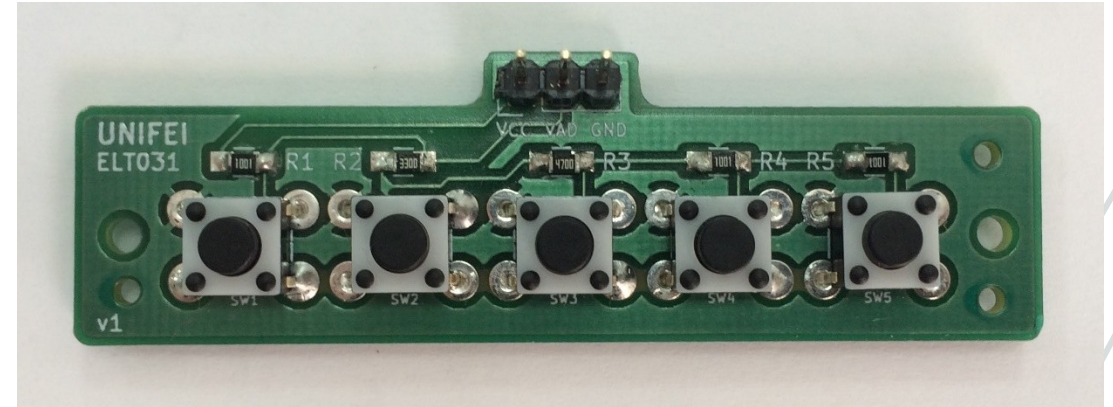
Falhou! Qual é o bug?

- Problema:
 - As teclas as vezes falham.
- Contexto:
 - O circuito do teclado é padronizado.
 - Foi utilizada uma biblioteca do ADC.



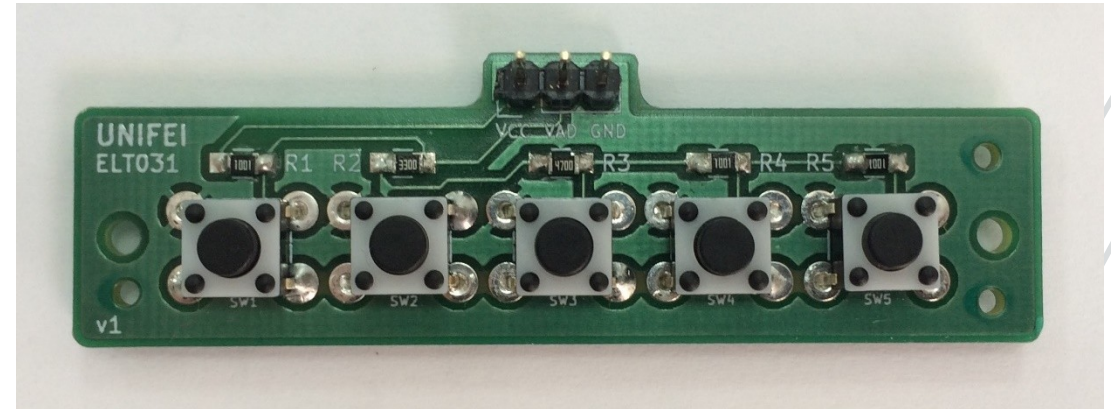
Qual o bug?

- Lib gerada errada?
 - Ler o datasheet.
 - Checar cada um dos bits de configuração.
- O pino é compartilhado com outros periféricos?
 - Ler o datasheet dos periféricos compartilhados.
 - Testar se suas configurações não estão atrapalhando o ADC.



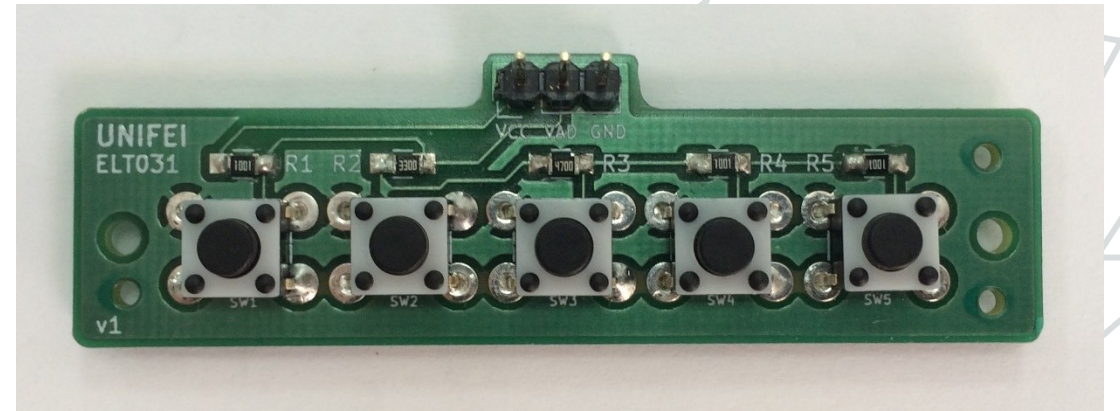
Qual o bug?

- Algum problema de temporização do loop principal?
- Criar um projeto só com o código do ADC e do teclado



Qual o bug?

- Montagem errada da placa?
 - As soldas estão bem feitas?
- Algum curto ou circuito errado?
- Nível de tensão bate com o previsto?
- Talvez algum resistor esteja com valor errado.

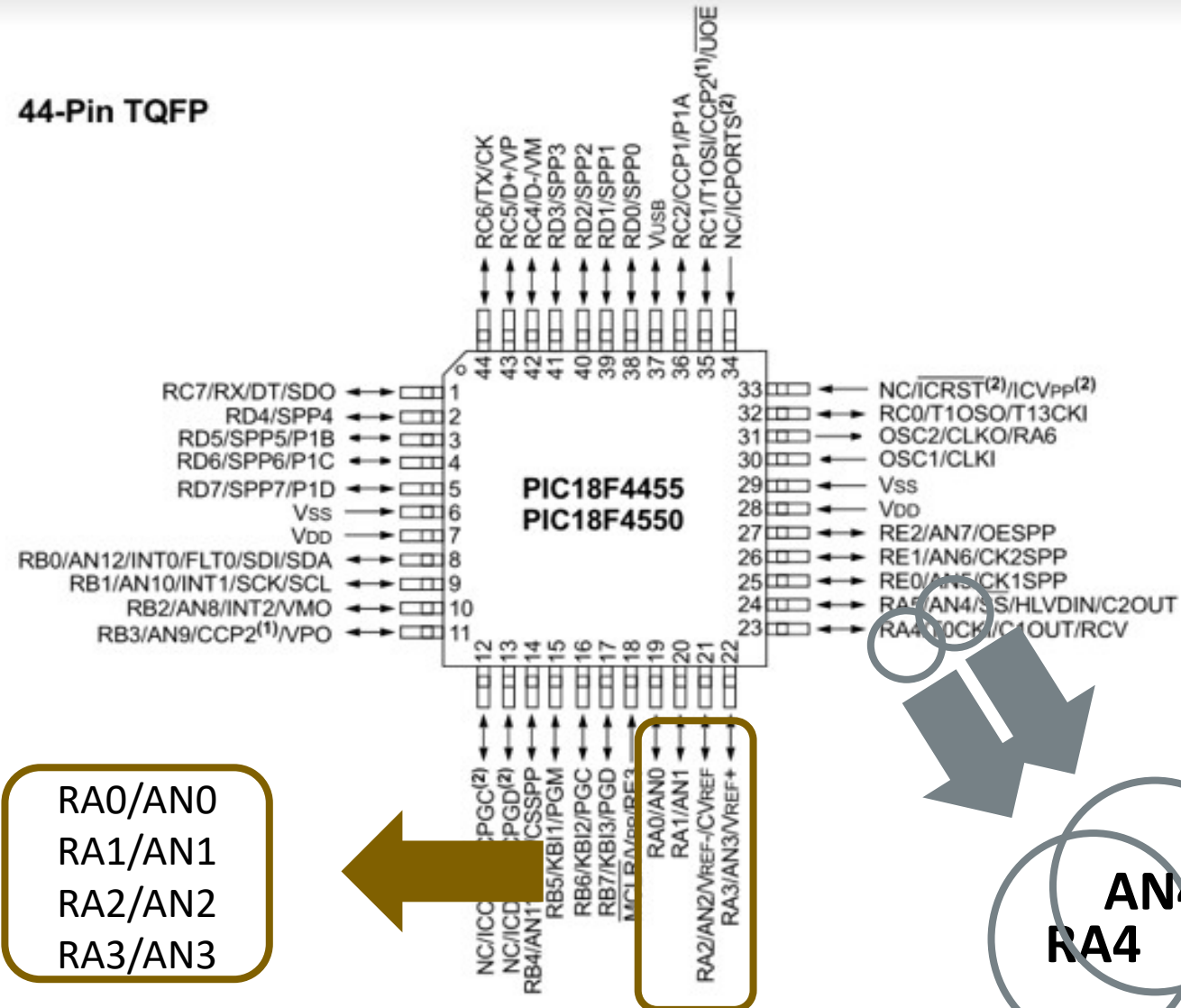


Encontrou?



O circuito - crosstalk

44-Pin TQFP

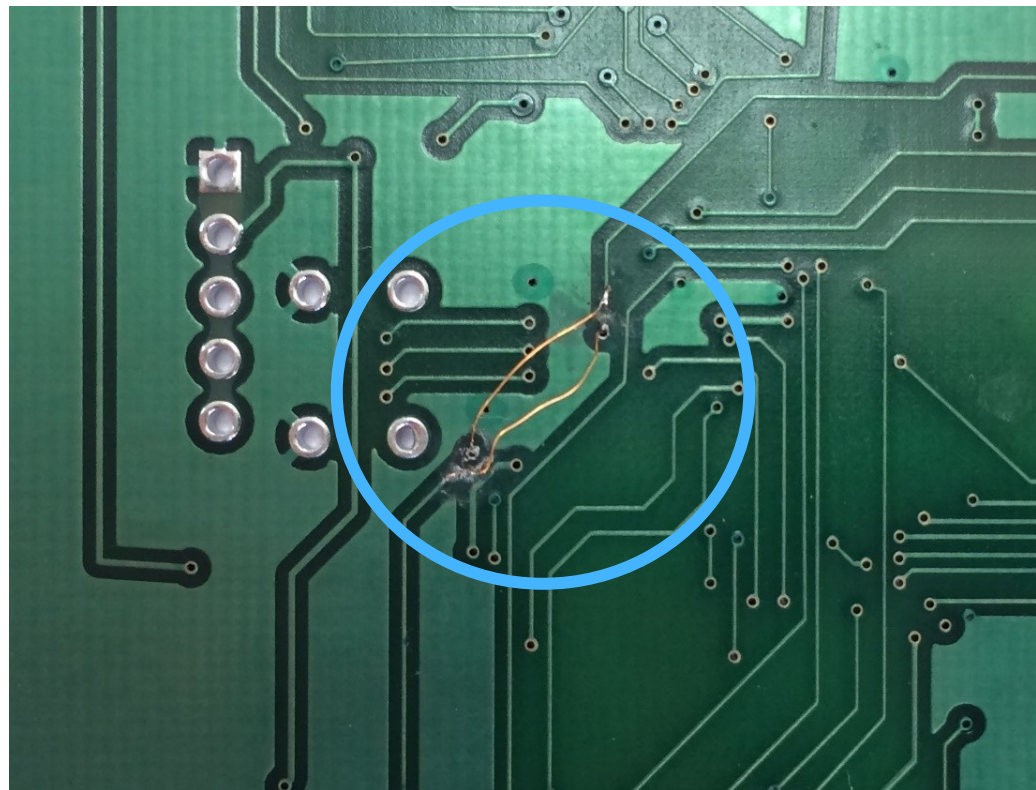


O problema foi encontrado

O pino do conversor AN4 foi confundido com a pino digital RA4.



A correção do bug



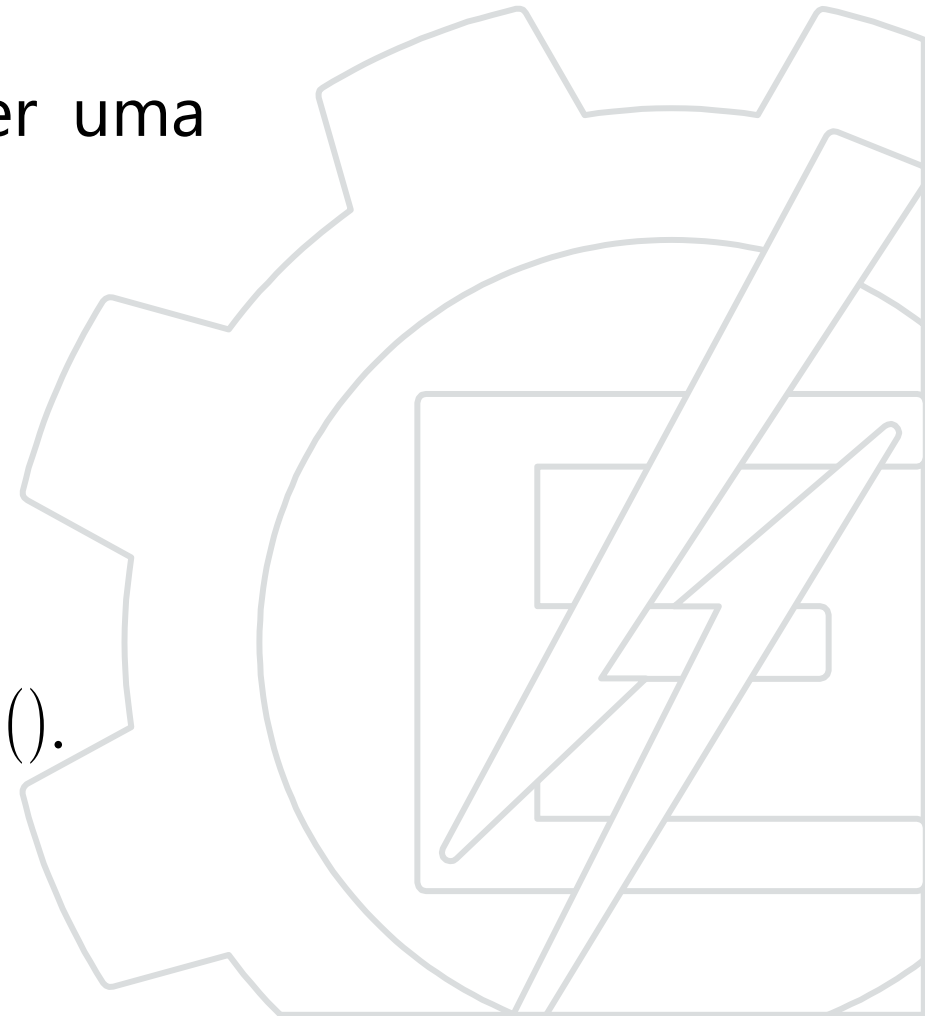
Em embarcados os Bugs não são só de software

Implementação e Teste



Implementação e Teste

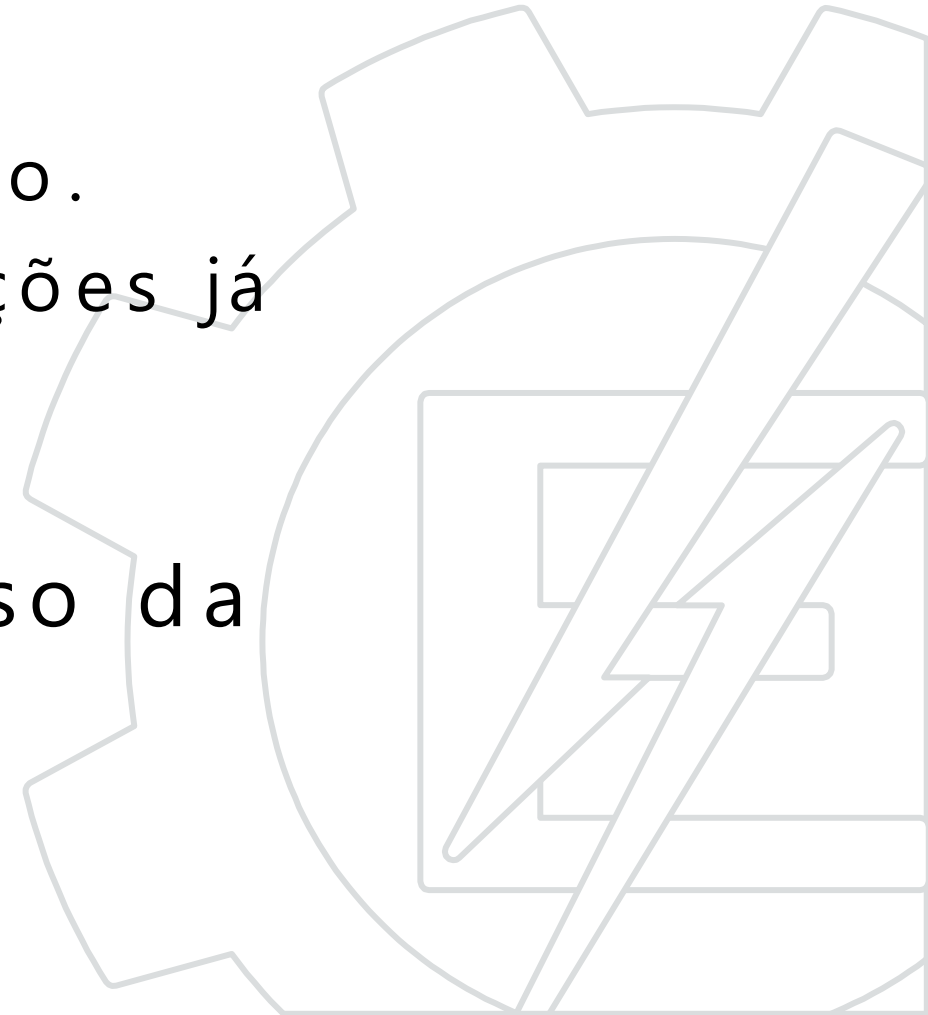
- Programação incremental.
 - Apenas inserir mais funções depois de ter uma primeira versão funcional.
 - Testar após cada alteração do código.
- Cuidado com alocações de memória.
 - Verificar se cada `malloc()` possui seu `free()`.



Implementação e Teste

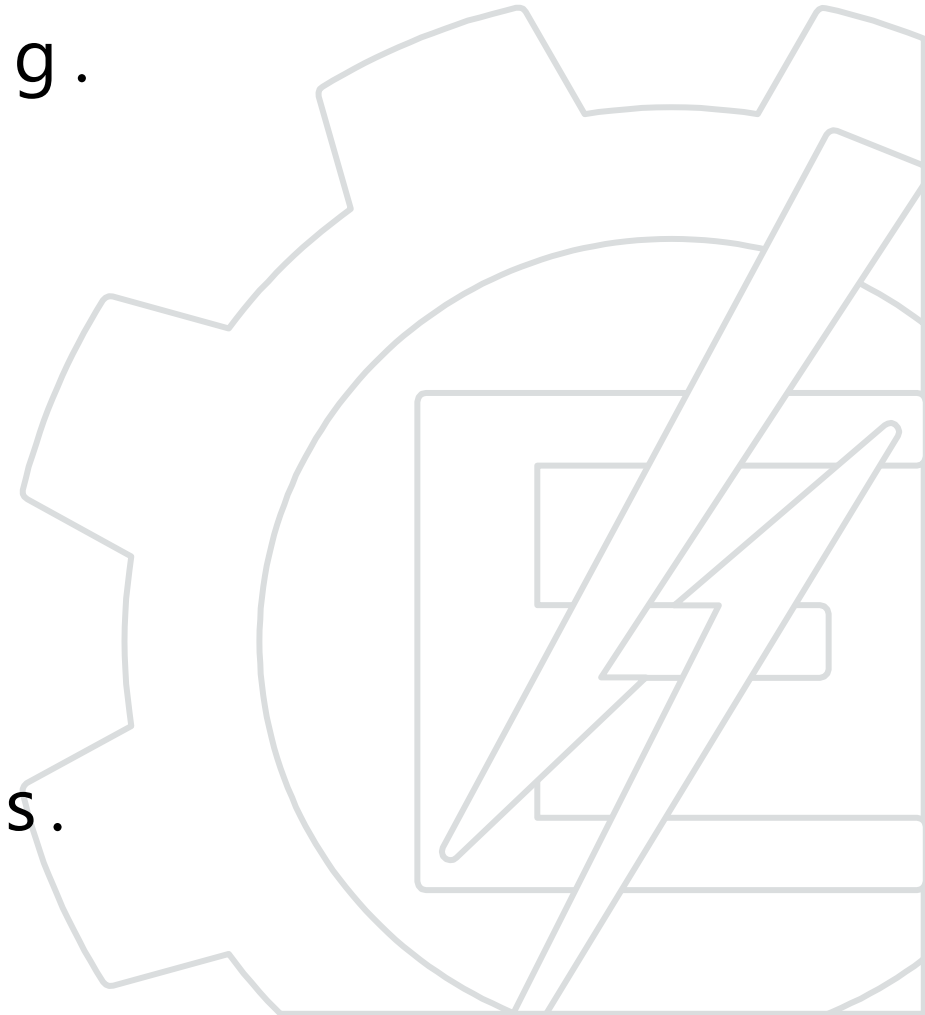
Otimização de código.

- Otimizar somente se necessário.
- Revisar o código e as otimizações já realizadas.
- Avaliar cuidadosamente o uso da memória e dos recursos.



Implementação e Teste

- Externalizar as informações.
 - Separar alguns leds para debug.
 - Enviar sinais via serial quando necessário.
 - Usar um emulador para obter informações.
- Reproduzir e isolar o erro.
 - Usar step-by-step de debug.
 - Isolar as funções problemáticas.



Até a próxima!

